Deep Learning Lecture Notes
2025/2026

João Pires

Instituto Superior Técnico, University of Lisbon, Portugal

January 21th, 2025

1. Revisão de Álgebra Linear (Lecture 1)

1.1. Conceitos e Operações Fundamentais

Uma matriz denotada por $A \in \mathbb{R}^{m \times n}$, é uma estrutura, regular, com m linhas e n colunas. Um vetor $x \in \mathbb{R}^n$ é um caso especial de uma matriz com apenas uma coluna, referido por vetor coluna. Se a matriz tiver apenas uma linha e várias colunas, chama-se de vetor linha.

A transposição de uma matriz, denotada por A^T, inverte a ordem dos elementos. Há uma troca entre as linhas e as colunas, fazendo com que (i, j) passe para (j, i). Se uma matriz se manter igual mesmo aposta a aplicação da transposição, dizemos que é uma matriz simétrica, $A^T = A$. O produto de matrizes, $C = AB$, constrói uma nova matriz onde cada elemento é a soma das multiplicações das linhas pelas colunas.

1.2. Outros Produtos Especializados

Um conceito importante é o produto interno entre dois vetores, denotado por $\langle x, y \rangle$ ou $x^T y$, onde x, y são vetores. Daqui resulta um número escalar. Também temos o produto externo, em que pega em dois vetores, $x \in \mathbb{R}^n$ e $y \in \mathbb{R}^m$, e produz uma matriz $n \times m$, onde cada elemento (i, j) é o produto de x_i por y_j. Por fim, temos o produto de Hadamard, denotado por $x \odot y$, que é uma multiplicação elemento a elemento que resulta em um vetor da mesma dimensão.

1.3. Propriedades das Operações Matriciais

A multiplicação de matrizes é associativa, significando que $(AB)C = A(BC)$. No entanto, não é comutativa em geral, ou seja, $AB \neq BA$ porque a ordem das transformações lineares importam. No quesito de regras para a transposição, temos que $(AB)^T = B^T A^T$ e que $(A + B)^T = A^T + B^T$.

1.4. Normas Vetoriais

Para medir o comprimento de um vetor, utilizamos normas. A norma mais comum, é a Euclidiana (Norma L_2), calculada como a raiz quadrada da soma dos quadrados das componentes do vetor. De forma genérica, a norma L_p é definida pela raiz p-ésima da soma das componentes do vetor elevados a p.

1.5. Matrizes Especiais

A matriz identidade, I, é uma matriz quadrada em que os únicos elementos não nulos que possuem estão na sua diagonal e são 1's. É o elemento neutro na multiplicação de matrizes. Existem as matrizes triangulares (superiores e inferiores), onde todos os elementos abaixo ou acima da diagonal, respetivamente, são zeros.

1.6. Valores Próprios, Traço, e Determinante

Uma análise mais profunda da matriz recorre aos seus valores e vetores próprios. Um vetor próprio, x, é um vetor que quando multiplicado pela matriz A não muda de direção, apenas de escala. E essa escala é dada pelo seu respetivo valor próprio, λ, temos que $Ax = \lambda x$. O traço de uma matriz é a soma dos elementos da diagonal principal, que também é igual à soma dos valores próprios da matriz. O determinante é o produto dos valores próprios e fornece uma medida de como a matriz se comporta no espaço. Se o determinante for zero, a matriz não é invertível.

1.7. Matriz Inversa

Uma matriz quadrada A é invertível (não singular) se existir uma matriz B tal que $AB = BA = I$, essa matriz B é chamada de inversa, A^{-1}. A condição para a existência da matriz inversa, é o determinante ser diferente de zero. Podemos resolver sistemas como, $Ax = b$, onde a solução é $x = A^{-1}b$. Outras propriedades são: $(A^{-1})^{-1} = A, \quad (A^{-1})^T = (A^T)^{-1}, \quad (AB)^{-1} = B^{-1}A^{-1}$.

1.8. Formas Quadráticas e Matrizes Definidas Positivas

Uma forma quadrática é uma função escalar construida a partir de uma matriz simétrica, A, e um vetor x, dada por $x^T Ax$. O sinal desta forma diz-nos muito sobre a geometria da matriz. Se $x^T Ax \geq 0$ para qualquer vetor x, a matriz é chamada de Positiva Semi-Definida (PSD), e temos que $\lambda_i(A) \geq 0$. Se $x^T Ax > 0$, ela é Positiva Definida (PD), e temos que $\lambda_i(A) > 0$.

2. Revisão de Probabilidade

2.1. Definições Básicas

A teoria da probabilidade atua como ponte entre os modelos probabilísticos (modelos teóricos) e os dados observados (dados reais). Existem 3 formas de definir a probabilidade: Clássica (baseada em resultados igualmente prováveis), Frequentista (baseada no limite da frequência relativa em infinitas tentativas) e a Subjetiva (baseada no grau de crença, inferência bayesiana). O espaço amostral, $\mathcal{X}$, representa todos os resultados possíveis, e um evento A é um subconjunto desse espaço. No fundo, temos isto:

- **Classical:** $P(A) = \frac{N_A}{N}$;
- **Frequentist:** $P(A) = \lim_{N\to\infty} \frac{N_A}{N}$;
- **Bayesian:** $P(A)$ is a degree of belief.

2.2. Axiomas e Teoremas Fundamentais

A probabilidade é comandada pelos Axiomas de Kolmogorov, que dizem que a probabilidade é não-negativa (For any A, $P(A) \geq 0$), a probabilidade do espaço amostral é 1 ($P(\mathcal{X}) = 1$), e a probabilidade da união de eventos disjuntos é a soma das suas probabilidades individuais ($P\left(\bigcup_i A_i\right) = \sum_i P(A_i)$). Outras relações que podemos tirar, são: $P(A \cup B) = P(A) + P(B) - P(A \cap B)$ e $P(\emptyset) = 0$. Também temos a probabilidade condicional $P(A|B)$, que atualiza a probabilidade de A considerando que B ocorreu. Se $P(A|B) = P(A)$, dizemos que os eventos são independentes. O Teorema de Bayes é uma ferramenta essencial para inverter as probabilidades condicionais, permitindo calcular $P(A_i|B)$ a partir de $P(B|A_i)$.

2.3. Variáveis Aleatórias Discretas e Contínuas

Uma variável aleatória (VA) mapeia resultados de um evento real para números reais. Para VAs Discretas, utilizamos a Função de Massa de Probabilidade (PMF), $f_X(x) = P(X = x)$, onde a soma de todas as probabilidades deve ser 1 ($F_X(x) = \sum_{x_i \leq x} f_X(x_i)$, onde $F_X(x)$ é a função de distribuição). Distribuições discretas importantes incluem:

- **Bernoulli:** Binário, 0 ou 1, $f_X(x) = p^x(1-p)^{1-x}$;
- **Binomial:** Soma de várias Bernoullis independentes, $f_X(x) = \binom{n}{x} p^x (1-p)^{n-x}$.

Para VAs Contínuas, utilizamos a Função de Densidade de Probabilidade (PDF), $f_X(x)$. Temos que:

$$F_X(x) = \int_{-\infty}^{x} f_X(u)\, du \quad , P(X \in [c,d]) = \int_{c}^{d} f_X(x)\, dx \quad , P(X = x) = 0.$$

Distribuições contínuas importantes incluem:

- **Uniform:** $f_X(x) = \text{Uniform}(x; a, b) = \begin{cases} \frac{1}{b-a}, \text{if } x \in [a,b] \\ 0, \text{if } x \notin [a,b] \end{cases}$
- **Gaussian:** $f_X(x) = \mathcal{N}(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$.

2.4. Expectativa e Variância

O valor esperado (ou Expectativa), $E(X)$, é a média ponderada dos valores possíveis da variável aleatória. Uma propriedade útil é a linearidade da expectativa, $E(X+Y) = E(X) + E(Y)$. A variância mede a dispersão dos dados em torno da média, e é calculada como $E[(X-\mu)^2]$. Temos, também, que:

$$E(X) = \begin{cases} \sum_i x_i f_X(x_i), & \text{if } X \in \{x_1, \ldots, x_K\} \subset \mathbb{R} \\ \int_{-\infty}^{\infty} x f_X(x)\, dx, & \text{if } X \text{ is continuous} \end{cases}.$$

Algumas relações importantes, em ter em causa:

- **Variance:** $\text{var}(X) = E\left[(X - E[X])^2\right] = E[X^2] - (E[X])^2$;
- **Bernoulli variance:** $E[X^2] = E[X] = p \implies \text{var}(X) = p - p^2 = p(1-p)$;
- **Gaussian variance:** $E\left[(X-\mu)^2\right] = \sigma^2$.

2.5. Probabilidade Multivariada e Independência

Quando temos múltiplas variáveis, utilizamos a sua distribuição conjunta $f_{X,Y}(x,y)$. Para duas variáveis discretas, temos $f_{X,Y}(x,y) = P(X = x \wedge Y = y)$. Para duas variáveis continuas, temos $f_{X,Y}(x,y) = \iint_A f_{X,Y}(x,y)\, dx\, dy$. A marginalização é o processo de recuperar a distribuição para apenas uma variável, integrando sobre todas as outras variáveis conjuntas:

$$f_Y(y) = \begin{cases} \sum_x f_{X,Y}(x,y), \text{if } X \text{ is discrete} \\ \int_{-\infty}^{\infty} f_{X,Y}(x,y)\, dx, \text{if } X \text{ is continuous} \end{cases}$$

A independência entre duas variáveis, X e Y, ocorre apenas se $f_{X,Y}(x,y) = f_X(x)f_Y(y)$. O que implica que $E(XY) = E(X)E(Y)$, mas $E(XY) = E(X)E(Y)$ não implica que $f_{X,Y}(x,y) = f_X(x)f_Y(y)$. Estendendo para vetores, temos a Multinomial (generalização da Binomial para K classes) e a Gaussiana Multivariada, parametrizada por um vetor de médias μ e uma matriz de covariância C.

$$\textbf{Multivariate Gaussian:} \quad f_{\mathbf{X}}(\mathbf{x}) = \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \mathbf{C}) = \frac{1}{\sqrt{\det(2\pi\mathbf{C})}} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T\mathbf{C}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right).$$

$$\textbf{Multinomial Coefficient:} \binom{n}{x_1, x_2, \ldots, x_K} = \frac{n!}{x_1! x_2! \cdots x_K!}.$$

2.6. Covariância e Correlação

A Covariância mede como duas variáveis variam juntas linearmente. Se a covariância é positiva, elas tendem a aumentar juntas; se negativa, uma aumenta enquanto a outra diminui. Se X e Y são independentes, a covariância é zero, mas o inverso não é necessariamente verdadeiro (podem ter dependência não linear). A covariância entre duas VAs, é dada por: $\text{cov}(X,Y) = E\left[(X - E[X])(Y - E[Y])\right] = E[XY] - E[X]E[Y]$. Relação entre a covariância e variância: $\text{var}(X) = \text{cov}(X,X)$. A matriz de covariância de um problema multivariado de VAs:

$$\text{Cov}(\mathbf{X}) = E\left[(\mathbf{X} - E[\mathbf{X}])(\mathbf{X} - E[\mathbf{X}])^T\right] = E[\mathbf{X}\mathbf{X}^T] - E[\mathbf{X}]E[\mathbf{X}]^T.$$

Seja $A \in \mathbb{R}^{n\times n}$ uma matriz e $\mathbf{a} \in \mathbb{R}^n$ um vetor:

- If $E[\mathbf{X}] = \boldsymbol{\mu}$ and $\mathbf{Y} = A\mathbf{X}$, then $E[\mathbf{Y}] = A\boldsymbol{\mu}$;
- If $\text{cov}(\mathbf{X}) = C$ and $\mathbf{Y} = A\mathbf{X}$, then $\text{cov}(\mathbf{Y}) = ACA^T$;
- If $\text{cov}(\mathbf{X}) = C$ and $Y = \mathbf{a}^T\mathbf{X} \in \mathbb{R}$, then $\text{var}(Y) = \mathbf{a}^T C\mathbf{a} \geq 0$;
- If $\text{cov}(\mathbf{X}) = C$ and $\mathbf{Y} = C^{-1/2}\mathbf{X}$, then $\text{cov}(\mathbf{Y}) = I$;
- If $f_{\mathbf{X}}(\mathbf{x}) = \mathcal{N}(\mathbf{x}; \mathbf{0}, I)$ and $\mathbf{Y} = \boldsymbol{\mu} + C^{1/2}\mathbf{X}$, then $f_{\mathbf{Y}}(\mathbf{y}) = \mathcal{N}(\mathbf{y}; \boldsymbol{\mu}, C)$;
- If $f_{\mathbf{X}}(\mathbf{x}) = \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, C)$ and $\mathbf{Y} = C^{-1/2}(\mathbf{X} - \boldsymbol{\mu})$, then $f_{\mathbf{Y}}(\mathbf{y}) = \mathcal{N}(\mathbf{y}; \mathbf{0}, I)$.

2.7. Entropia e Divergência KL

A entropia $H(X)$ de uma VA discreta, X, quantifica a sua incerteza associada. E é dada por: $H(X) = -\sum_{x=1}^{K} f_X(x) \log f_X(x)$. Para um VA contínua, temos que: $h(X) = -\int_{-\infty}^{\infty} f_X(x) \log f_X(x)\, dx$. Quanto mais imprevisível a distribuição, maior a sua entropia. A divergência de Kullback-Leibler (KLD), denotada por $D(f_X || g_X)$, mede a diferença entre duas distribuições de probabilidade f e g. E é dado por, $D(f_X \parallel g_X) = \sum_{x=1}^{K} f_X(x) \log \frac{f_X(x)}{g_X(x)}$.

3. Revisão de Optimização

3.1. Conceitos Iniciais

O nosso principal objetivo é encontrar um conjunto de parâmetros, normalmente denominado por um vetor x^*, que minimize a nossa função de custo $f : \mathbb{R}^n \to \mathbb{R}$. Queremos encontrar um mínimos global, i.e, um ponto tal que: $\forall \mathbf{x} \in \mathbb{R}^n,\ f(\mathbf{x}^*) \leq f(\mathbf{x})$. Mas muitas vezes, devido a não sabermos como a nossa função se comporta em todo o espaço (funções complexas), temos apenas mínimos locais. Isto é, temos algo do tipo: $\|\mathbf{x} - \mathbf{x}^*\| \leq \delta \implies f(\mathbf{x}^*) \leq f(\mathbf{x})$. São pontos onde a função é mais baixa, apenas numa vizinhança definida num raio δ.

3.2. Funções Convexas

Para analisarmos o comportamento de uma função, olhamos para a sua convexidade. Uma função é convexa se, ao traçarmos um segmento de reta entre quaisquer dois pontos da curva, esse segmento permanecer acima ou sobre a função. Se o segmento estiver estritamente acima da função (sem tocar, exceto nas extremidades), a função é estritamente convexa.

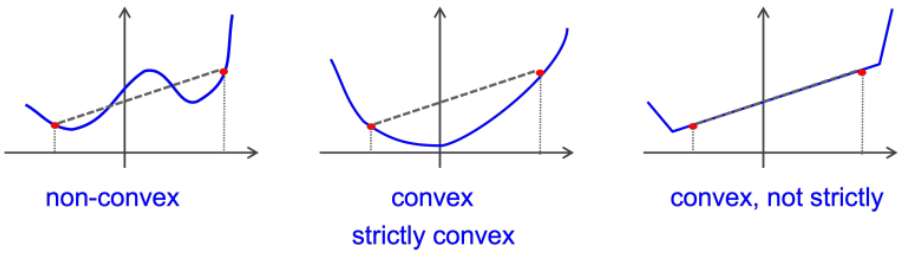

Figure 1: Funções convexas, e os seus tipos.

3.3. Relação entre Convexidade e Minimização

Sabendo a convexidade da função, o problema de optimização fica mais simples. Se a função f for convexa, qualquer mínimo que encontremos é garantidamente um mínimo global. Se a função for estritamente convexa, esse mínimo global é único.

3.4. Gradientes e Minimização

Para encontrar mínimos em funções diferenciáveis, utilizamos o gradiente $\nabla f(x)$, que é o vetor de derivadas parciais apontando na direção de maior subida da função. Uma condição necessária para um ponto x^* ser um mínimo local é que o gradiente seja zero ($\nabla f(x^*) = 0$). No entanto, gradiente zero não garante um mínimo, pode ser também um máximo local ou um ponto de sela. No que toca à matriz hessiana, $\nabla^2 f(x)$, se a hessiana for uma matriz positiva semi-definida, temos uma função convexa. Se for positiva-definida, temos uma função estritamente convexa.

- **Quadratic Form (General):**

$$f(\mathbf{x}) = \mathbf{x}^T A \mathbf{x} : \quad \nabla f(\mathbf{x}) = (A + A^T)\mathbf{x}$$

- **Quadratic Form (Symmetric):**

$$\text{If } A \text{ is symmetric} : \quad \nabla f(\mathbf{x}) = 2A\mathbf{x}$$

- **Squared Norm:**

$$f(\mathbf{x}) = \mathbf{x}^T\mathbf{x} = \|\mathbf{x}\|_2^2, \quad \text{then } \nabla f(\mathbf{x}) = 2\mathbf{x}$$

- **Linear Form:**

$$\text{If } f(\mathbf{x}) = \mathbf{x}^T\mathbf{b} = \mathbf{b}^T\mathbf{x}, \quad \text{then } \nabla f(\mathbf{x}) = \mathbf{b}$$

- **Chain Rule (Linear):**

$$\text{If } g(\mathbf{x}) = f(A\mathbf{x}), \quad \text{then } \nabla g(\mathbf{x}) = A^T\nabla f(A\mathbf{x})$$

- **Chain Rule (Element-wise/Hadamard):**

$$\text{If } g(\mathbf{x}) = f(\mathbf{a} \odot \mathbf{x}), \quad \text{then } \nabla g(\mathbf{x}) = \mathbf{a} \odot \nabla f(\mathbf{a} \odot \mathbf{x})$$

- **Least Squares (Analytical Minima):**

$$\begin{aligned} f(\mathbf{x}) &= \|A\mathbf{x} - \mathbf{y}\|_2^2 \\ \nabla f(\mathbf{x}) &= 2A^T(A\mathbf{x} - \mathbf{y}) = \mathbf{0} \\ \implies \quad \mathbf{x}^* &= (A^TA)^{-1}A^T\mathbf{y} \end{aligned}$$

3.5. Gradient Descent

Quando a solução analítica é computacionalmente inviável, usamos métodos iterativos como o Gradient Descent. A ideia é começar num ponto aleatório $x^{(0)}$ e dar passos sucessivos na direção oposta ao gradiente: $x^{(t+1)} \leftarrow x^{(t)} - \eta\nabla f(x^{(t)})$. O hiperparâmetro η, ou taxa de aprendizagem (step-size), é crucial: se for muito pequeno, a convergência é lenta; se for muito grande, o algoritmo pode divergir e nunca encontrar o mínimo.

4. Lecture 2: Linear Classifiers I

4.1. A Relevância dos Classificadores Lineares

Começamos a falar de linear classifiers, apesar de se tratar de uma UC de Deep Learning, porque, no fundo, os conceitos de machine learning permanecem os mesmos. Também porque a estatística e a otimização são campos bem estudados, e os linear classifiers são muito utilizados (especialmente quando os dados são escassos). E, também, os linear classifiers são componentes que compõem as neural networks que, por sua vez, são compostas por camadas de transformações lineares seguidas de não-lineares.

4.2. Conexão com Neural Networks

Imaginemos uma neural network criada para distinguir imagens de animais entre as classes 'cão' e 'gato'. Se nos focarmos apenas na última camada (camada de output), verificamos que se trata de um linear classifier que tem um hiperplano linear bem definido entre as duas classes. Portanto, entender os linear classifiers é essencial para entender toda a neural network em si.

4.3. Feature Maps

Feature engineering é um passo essencial no funcionamento dos linear classifiers. Isto porque um modelo raramente opera sobre os dados brutos, normalmente há uma definição de um feature map, denotado matematicamente como $\phi : \mathcal{X} \to \mathbb{R}^D$. Esta função transforma a entrada bruta de x num vetor de atributos $\phi(x)$ com dimensão D. Este vetor pode conter características booleanas, categóricas ou contínuas. Atributos categóricos (como "dia da semana") são frequentemente convertidos em vetores binários "one-hot", onde apenas a posição correspondente à categoria ativa é 1 e as restantes são 0.

4.4. Feature Engineering

Antes de termos deep learning, existia um processo muito importante chamado de feature engineering. Este passo tinha como principal objetivo facilitar o trabalho do linear classifier, entregando-lhe os dados já 'mastigados' e relevantes, para que fizesse a melhor previsão possível. Em Processamento de Linguagem Natural (NLP), isto envolvia criar vetores baseados na ocorrência de palavras (Bag-of-Words), lemas ou classes gramaticais (part-of-speech). Em Visão Computacional, usavam-se descritores manuais complexos como SIFT ou gradientes de imagem para identificar arestas e texturas.

4.5. Pipelines Clássicos de NLP

Os sistemas clássicos de NLP eram construídos empilhando vários classificadores lineares em sequência, formando um "pipeline". A saída de um classificador servia como atributo de entrada para o seguinte. Por exemplo, primeiro classificava-se a função gramatical de cada palavra (se é verbo ou nome), e essa informação era usada num modelo subsequente para tarefas mais complexas.

4.6. Engineering vs. Learning

A feature engineering é frequentemente descrita como uma "alquimia". Ela exige um conhecimento profundo do domínio e consome muito tempo. Ao desenharmos atributos manualmente, estamos a introduzir viés indutivo (inductive bias) no sistema, ou seja, estamos a dizer ao modelo o que nós achamos que é importante. Embora trabalhosa, esta abordagem ainda é crucial em cenários onde temos poucos dados, pois o conhecimento humano compensa a falta de exemplos. A alternativa moderna, que é o cerne do Deep Learning, é a aprendizagem de representações (representation learning), onde o modelo aprende a construir o seu próprio vetor $\phi(x)$ diretamente dos dados, dispensando a engenharia manual.

5. Regressão Linear: O Básico

Diferente da classificação, onde o objetivo é prever uma categoria discreta, na regressão o espaço de saída é contínuo (por exemplo, prever o tempo de leitura de um artigo com base no número de palavras). O modelo linear assume que a saída $\hat{y}$ é uma combinação linear dos atributos de entrada x, ponderados por um vetor de pesos w e ajustados por um viés b. Para simplificar a matemática, frequentemente incorporamos o viés b dentro do vetor de pesos w, adicionando um atributo constante "1" ao vetor de características $\phi(x)$.

Muitas vezes, assumir uma dependência linear de y em relação a x é uma simplificação inadequada. Um modelo mais geral utiliza uma função de base para transformar o espaço de entrada:

$$y = \mathbf{w}^T \phi(x).$$

Onde $\phi(x)$ é um feature vector. Por exemplo, para um modelo polinomial de grau D:

$$\phi(x) = [1, x, x^2, \ldots, x^D]^T \in \mathbb{R}^{D+1}.$$

O termo de bias (b) é capturado pela feature constante $\phi_0(x) = 1$. O objetivo é ajustar ou aprender os parâmetros $\mathbf{w}$ resolvendo o problema de optimização:

$$\min_{\mathbf{w} \in \mathbb{R}^{D+1}} \sum_{n=1}^{N} \left(y_n - \mathbf{w}^T \phi(x_n)\right)^2, \quad \mathbf{X} = \begin{bmatrix} \phi(x_1)^T \\ \vdots \\ \phi(x_n)^T \\ \vdots \\ \phi(x_N)^T \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \\ \vdots \\ y_N \end{bmatrix}.$$

Sendo a closed-form solution:

$$\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}. \tag{1}$$

A matriz X é designada de design matrix. Este modelo continua a ser designado por regressão linear, uma vez que a linearidade é definida em relação aos parâmetros do modelo $\mathbf{w}$, e não necessariamente em relação às variáveis de entrada x.

A demonstração é dada por:

$$\sum_{n=1}^{N} (y_n - \mathbf{w}^T \phi(x_n))^2 = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2.$$

Para minimizar esta função, igualamos o gradiente em relação a $\mathbf{w}$ a zero:

$$0 = \nabla_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2.$$

Expandindo a norma quadrática através de identidades de álgebra linear:

$$0 = \nabla_{\mathbf{w}} \left(\mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w} - 2\mathbf{w}^T \mathbf{X}^T \mathbf{y} + \|\mathbf{y}\|^2\right).$$

Aplicando as regras de derivação matricial ($\frac{\partial \mathbf{a}^T \mathbf{x}}{\partial \mathbf{x}} = \mathbf{a}$ e $\frac{\partial \mathbf{x}^T \mathbf{A} \mathbf{x}}{\partial \mathbf{x}} = 2\mathbf{A}\mathbf{x}$ para matrizes simétricas):

$$0 = 2\mathbf{X}^T \mathbf{X} \mathbf{w} - 2\mathbf{X}^T \mathbf{y}.$$

Resolvendo a equação resultante para $\mathbf{w}$:

$$\mathbf{X}^T \mathbf{X} \mathbf{w} = \mathbf{X}^T \mathbf{y}.$$

Portanto, a solução de forma fechada é:

$$\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}.$$

5.1. Interpretação Probabilística: MLE

Por que razão escolhemos minimizar o erro ao quadrado e não o valor absoluto ou outra métrica? Se assumirmos que os nossos dados são gerados por uma função linear somada a um ruído Gaussiano, dada por:

$$y_n = \mathbf{w}_*^T \phi(x_n) + z_n.$$

Onde, $\mathbf{w}_*$ representa o vetor de parâmetros "verdadeiro" do modelo e $z_n \sim \mathcal{N}(0, \sigma^2)$ são amostras de ruído Gaussiano. A probabilidade de observar os dados y dado os pesos w segue uma distribuição normal:

$$y_n \sim \mathcal{N}(\mathbf{w}_*^T \phi(x_n), \sigma^2).$$

Quando aplicamos a Maximum Likelihood Estimation (MLE), que procura os parâmetros que tornam os dados observados o mais prováveis possível, a matemática simplifica-se. Maximizar a probabilidade dos dados sob ruído Gaussiano é matematicamente equivalente a minimizar o erro quadrático.

Recorde-se a função de densidade de probabilidade da distribuição Normal:

$$\mathcal{N}(y; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y-\mu)^2}{2\sigma^2}\right).$$

O estimador de máxima verosimilhança $\hat{\mathbf{w}}_{\text{MLE}}$ é derivado da seguinte forma:

$$\hat{\mathbf{w}}_{\text{MLE}} = \arg\max_{\mathbf{w}} P(\mathbf{y}|\mathbf{w}).$$

Assumindo que as amostras são independentes e identicamente distribuídas (i.i.d.), a verosimilhança conjunta é o produto das verosimilhanças individuais. Aplicando o logaritmo (que é uma função monótona crescente):

$$\hat{\mathbf{w}}_{\text{MLE}} = \arg\max_{\mathbf{w}} \log \prod_{n=1}^{N} P(y_n|\mathbf{w}),$$

$$\hat{\mathbf{w}}_{\text{MLE}} = \arg\max_{\mathbf{w}} \sum_{n=1}^{N} \log P(y_n|\mathbf{w}).$$

Substituindo pela densidade da Gaussiana:

$$\hat{\mathbf{w}}_{\text{MLE}} = \arg\max_{\mathbf{w}} \sum_{n=1}^{N} \left[-\frac{(y_n - \mathbf{w}^T\phi(x_n))^2}{2\sigma^2} - \underbrace{\log(\sqrt{2\pi\sigma^2})}_{\text{constante}} \right].$$

Como queremos maximizar o valor e o termo constante não depende de $\mathbf{w}$, podemos ignora-lo. Além disso, maximizar um valor negativo é equivalente a minimizar o seu valor absoluto:

$$\hat{\mathbf{w}}_{\text{MLE}} = \arg\min_{\mathbf{w}} \sum_{n=1}^{N} (y_n - \mathbf{w}^T\phi(x_n))^2.$$

A regressão linear com perda quadrática (squared loss) é equivalente à Estimativa de Máxima Verosimilhança sob a premissa de ruído Gaussiano.

5.2. Loss Functions na Regressão

Embora o erro quadrático seja padrão, ele não é o único. Ele é muito sensível a outliers, pois eleva o erro ao quadrado (um erro grande torna-se enorme). Erro Quadrático (Squared loss):

$$L(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2.$$

Penaliza o erro linearmente ($|y - \hat{y}|$). É mais robusta a outliers, mas tem problemas de otimização porque não é diferenciável no zero. Erro Absoluto (Absolute error loss):

$$L(y, \hat{y}) = |y - \hat{y}|.$$

Comporta-se como erro quadrático para erros pequenos (perto de zero, onde precisamos de precisão e diferenciabilidade) e como erro absoluto para erros grandes (para não dar peso excessivo a outliers). Huber loss:

$$L(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{se } |y - \hat{y}| \leq 1, \\ |y - \hat{y}| - \frac{1}{2} & \text{se } |y - \hat{y}| > 1. \end{cases}$$

Todas as funções acima são convexas. Isto é excelente para otimização, pois garante que qualquer mínimo local é um mínimo global. Apenas a Squared Loss é estritamente convexa. Isso garante que, se existir uma solução para o problema de mínimos quadrados, ela será única.

5.3. Overfitting e Underfitting

O Underfitting ocorre quando o modelo é demasiado simples (ex: tentar ajustar uma curva complexa com uma reta simples, $D = 1$). O modelo não captura a tendência dos dados. Overfitting ocorre quando o modelo é demasiado complexo (ex: polinómio de grau alto). O modelo passa por quase todos os pontos de treino, mas oscila violentamente, capturando o "ruído" em vez do sinal real.

5.4. Ridge Regression e Regularization

O overfitting resulta frequentemente em pesos w com valores muito grandes para compensar oscilações nos dados. Como vimos anteriormente, a solução de mínimos quadrados (LS) tem uma forma fechada definida por:

$$\hat{\mathbf{w}}_{LS} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}.$$

Existem casos em que a matriz $\mathbf{X}^T\mathbf{X}$ não é invertível (singular). Isto acontece frequentemente quando temos características colineares (redundância nos dados), e/ou o número de características D é superior ao número de amostras N ($D > N$). A abordagem padrão para resolver este problema é a Regressão de Ridge, que adiciona um termo à diagonal da matriz antes da inversão:

$$\hat{\mathbf{w}}_{\text{ridge}} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{y}.$$

Onde $\lambda > 0$ é um hiperparâmetro de regularização e $\mathbf{I}$ é a matriz identidade. ste estimador é a solução exata para o seguinte problema de minimização, utilizando a norma ℓ_2 ao quadrado ($\|\mathbf{w}\|_2^2 = \sum_i w_i^2$):

$$\hat{\mathbf{w}}_{\text{ridge}} = \arg\min_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 + \lambda\|\mathbf{w}\|_2^2.$$

A regularização ℓ_2 é também conhecida em diferentes contextos como Weight Decay e Penalized LS.

5.5. Maximum A Posteriori (MAP)

Tal como os Mínimos Quadrados correspondem ao MLE, a Ridge Regression também tem uma interpretação probabilística Bayesiana. Em vez de assumirmos que não sabemos nada sobre os pesos w, introduzimos uma distribuição a priori (Prior), $\mathbf{w} \sim \mathcal{N}(0, \tau^2 I)$. O critério MAP procura o valor de $\mathbf{w}$ mais provável dada a evidência dos dados $\mathbf{y}$:

$$\hat{\mathbf{w}}_{\text{MAP}} = \arg\max_{\mathbf{w}} P(\mathbf{w}|\mathbf{y}).$$

Utilizando a Regra de Bayes ($P(\mathbf{w}|\mathbf{y}) = \frac{P(\mathbf{y}|\mathbf{w})P(\mathbf{w})}{P(\mathbf{y})}$) e ignorando o denominador $P(\mathbf{y})$ (que não depende de $\mathbf{w}$):

$$\hat{\mathbf{w}}_{\text{MAP}} = \arg\max_{\mathbf{w}} P(\mathbf{w}) \prod_{n=1}^{N} P(y_n|\mathbf{w}).$$

Aplicando o logaritmo para facilitar o cálculo:

$$\hat{\mathbf{w}}_{\text{MAP}} = \arg\max_{\mathbf{w}} \left(\log P(\mathbf{w}) + \sum_{n=1}^{N} \log P(y_n|\mathbf{w}) \right).$$

Substituindo pelas densidades das distribuições Gaussianas:

$$\hat{\mathbf{w}}_{\text{MAP}} = \arg\max_{\mathbf{w}} \left(-\frac{\|\mathbf{w}\|^2}{2\tau^2} - \sum_{n=1}^{N} \frac{(y_n - \mathbf{w}^T\phi(x_n))^2}{2\sigma^2} + \text{constante} \right).$$

Para transformar isto num problema de minimização, multiplicamos por -1 (e para simplificar, multiplicamos por $2\sigma^2$):

$$\hat{\mathbf{w}}_{\text{MAP}} = \arg\min_{\mathbf{w}} \left(\underbrace{\frac{\sigma^2}{\tau^2}\|\mathbf{w}\|^2}_{\text{regularizador}} + \underbrace{\sum_{n=1}^{N} (y_n - \mathbf{w}^T\phi(x_n))^2}_{\text{perda (loss)}} \right).$$

Definindo $\lambda = \sigma^2/\tau^2$ como a constante de regularização, obtemos a formulação de Ridge Regression.

6. Binary Classification e Hiperplano

Ao contrário da regressão, onde prevemos um valor contínuo, na classificação binária o objetivo é atribuir uma etiqueta y que pertence a um conjunto de duas classes, tipicamente $\{-1, +1\}$. Para adaptar o modelo linear a esta tarefa, mantemos o cálculo de um score $(w^T\phi(x) + b)$, mas aplicamos uma função de sinal (sign). Se a pontuação for positiva ou zero, prevemos $+1$; se for negativa, prevemos -1:

$$\hat{y} = \text{sign}(\mathbf{w}^T\phi(x) + b) = \begin{cases} +1, & \text{se } \mathbf{w}^T\phi(x) + b \geq 0, \\ -1, & \text{se } \mathbf{w}^T\phi(x) + b < 0. \end{cases}$$

Geometricamente, isto significa que o modelo define um hiperplano de decisão no espaço de atributos. Este hiperplano divide o espaço em duas metades: os pontos de um lado são classificados como positivos e os do outro como negativos. Os pontos que caem exatamente sobre a linha têm pontuação zero. A separação entre classes ocorre exatamente onde o score é zero. Esta fronteira é um hiperplano no espaço das features $\phi(x)$:

$$\mathbf{w}^T \phi(x) + b = 0.$$

6.1. Separabilidade Linear

Um conceito fundamental para estes modelos é a separabilidade linear. Dizemos que um conjunto de dados é linearmente separável se existir, de facto, um hiperplano (definido por pesos w e viés b) que consiga classificar perfeitamente todos os exemplos de treino sem erros. Se os dados estiverem misturados de forma complexa (por exemplo, um círculo de pontos positivos rodeado por pontos negativos), um classificador linear simples nunca conseguirá obter uma precisão perfeita. Para simplificar a matemática nestes modelos, é comum remover o termo de viés b explícito, adicionando um atributo constante com valor 1 ao vetor de características $\phi(x)$, integrando o viés no vetor de pesos w.

6.2. Perceptron

O Perceptron, é um algoritmo online, o que significa que processa um exemplo de treino de cada vez. A lógica é baseada em erros: o algoritmo percorre os dados e faz uma previsão. Se a previsão estiver correta, não faz nada. Se estiver errada, atualiza o vetor de pesos w. A regra de atualização é simples: se o modelo errou num exemplo positivo (previu negativo), ele soma o vetor de atributos desse exemplo aos pesos; se errou num negativo, subtrai-o. Intuitivamente, isto "roda" o hiperplano ligeiramente na direção correta para tentar classificar esse exemplo acertadamente na próxima vez.

6.3. Perceptron's Mistake Bound

Para analisar a eficiência do algoritmo Perceptron, utilizamos duas definições fundamentais. Os dados de treino são linearmente separáveis com margem $\gamma > 0$ se, e somente se, existir um vetor de pesos ótimo $\mathbf{u}$, com $\|\mathbf{u}\| = 1$, tal que:

$$y_n(\mathbf{u}^T \phi(x_n)) \geq \gamma, \quad \forall n.$$

Raio dos Dados (R): É a magnitude máxima observada no vetor de features:

$$R = \max_n \|\phi(x_n)\|.$$

O algoritmo Perceptron tem a garantia de encontrar um hiperplano separador após, no máximo, o seguinte número de erros. Theorem (Novikoff (1962)) :

$$k \leq \frac{R^2}{\gamma^2}.$$

Recorde-se a regra de atualização do Perceptron: sempre que ocorre um erro na amostra n, atualizamos os pesos como $\mathbf{w}^{(k)} = \mathbf{w}^{(k-1)} + y_n \phi(x_n)$. Analisamos o crescimento de $\mathbf{w}^{(k)}$ na direção do vetor unitário ótimo $\mathbf{u}$ (onde $y_n \mathbf{u}^T \phi(x_n) \geq \gamma$):

$$\mathbf{u}^T \mathbf{w}^{(k)} = \mathbf{u}^T (\mathbf{w}^{(k-1)} + y_n \phi(x_n)) = \mathbf{u}^T \mathbf{w}^{(k-1)} + y_n \mathbf{u}^T \phi(x_n),$$

$$\mathbf{u}^T\mathbf{w}^{(k)} \geq \mathbf{u}^T\mathbf{w}^{(k-1)} + \gamma.$$

Aplicando recursivamente a partir de $\mathbf{w}^{(0)} = 0$, após k erros temos:

$$\mathbf{u}^T\mathbf{w}^{(k)} \geq k\gamma.$$

Pela desigualdade de Cauchy-Schwarz (e sabendo que $\|\mathbf{u}\| = 1$). Temos a **Lower Bound**:

$$\|\mathbf{w}^{(k)}\| \geq \mathbf{u}^T\mathbf{w}^{(k)} \implies \|\mathbf{w}^{(k)}\| \geq k\gamma.$$

Analisamos o crescimento da norma quadrada. Atualização ocorre se $y_n(\mathbf{w}^{(k-1)})^T\phi(x_n) \leq 0$:

$$\|\mathbf{w}^{(k)}\|^2 = \|\mathbf{w}^{(k-1)} + y_n\phi(x_n)\|^2 = \|\mathbf{w}^{(k-1)}\|^2 + \underbrace{\|y_n\phi(x_n)\|^2}_{\leq R^2} + \underbrace{2y_n(\mathbf{w}^{(k-1)})^T\phi(x_n)}_{\leq 0 \text{ (condição de erro)}},$$

$$\|\mathbf{w}^{(k)}\|^2 \leq \|\mathbf{w}^{(k-1)}\|^2 + R^2.$$

Aplicando recursivamente. Temos a **Upper Bound**:

$$\|\mathbf{w}^{(k)}\|^2 \leq kR^2.$$

Combinando os dois limites:

$$(k\gamma)^2 \leq \|\mathbf{w}^{(k)}\|^2 \leq kR^2,$$

$$k^2\gamma^2 \leq kR^2 \implies k \leq \frac{R^2}{\gamma^2}.$$

6.4. Limitações

Apesar das garantias teóricas, o Perceptron tem uma limitação fatal: só funciona se a fronteira de decisão for linear. O exemplo clássico onde falha é o problema XOR (Ou Exclusivo). Imagine quatro pontos num gráfico: $(0, 0)$ e $(1, 1)$ são de uma classe, e $(0, 1)$ e $(1, 0)$ são de outra. É impossível desenhar uma única linha reta que separe estas classes corretamente.

6.5. Multi-Class Classification

Muitos problemas reais envolvem mais do que duas classes (ex: classificar dígitos de 0 a 9). Existem estratégias para reduzir isto a problemas binários, como treinar um classificador para cada classe contra todas as outras (One-vs-All) ou treinar pares de classificadores (One-vs-One). No entanto, podemos generalizar o modelo linear para lidar diretamente com K classes. Em vez de um vetor de pesos, temos uma matriz de pesos W, onde cada linha corresponde a uma classe.

6.6. Perceptron Multi-classe

Nesta formulação direta, calculamos uma pontuação para cada uma das classes possíveis multiplicando a entrada pelos pesos correspondentes. A previsão é simplesmente a classe que obtiver a pontuação mais alta (argmax). O algoritmo Perceptron adapta-se facilmente a este cenário: se prevermos a classe errada, subtraímos o vetor de atributos dos pesos da classe errada (para diminuir a sua pontuação futura) e somamo-lo aos pesos da classe correta (para aumentar a sua pontuação futura). Geometricamente, isto divide o espaço em regiões convexas, onde cada região pertence a uma classe.

7. Lecture 3: Linear Classifiers II

7.1. Transformação Softmax e Sigmoid

Anteriormente, calculávamos uma score (z) para cada classe. No entanto, pontuações brutas (que variam de $-\infty$ a $+\infty$) são difíceis de interpretar. O objetivo principal aqui é transformar esses valores numa distribuição de probabilidade válida (valores entre 0 e 1 que somam 1). Para classificação binária, utilizamos a transformação Sigmoid (σ), que comprime qualquer valor real para o intervalo $[0, 1]$. Já para a classificação multi-classe, utilizamos a função Softmax, que generaliza a Sigmoid, normalizando o vetor de pontuações de modo a que a soma de todas as probabilidades das classes seja igual a 1.

7.2. Logistic Regression

A Regressão Logística utiliza estas transformações para criar um classificador probabilístico. Apesar do nome "regressão", é um modelo de classificação. O objetivo do treino não é minimizar o erro quadrático, mas sim maximizar a verosimilhança condicional (Conditional Log-Likelihood). Intuitivamente, queremos encontrar os pesos W que atribuem a maior probabilidade possível às labels verdadeiras observadas nos dados de treino. Matematicamente, maximizar a probabilidade é equivalente a minimizar a Log-Likelihood Negativa (também conhecida como entropia cruzada).

Transformação Softmax

Num modelo probabilístico linear, associamos a cada classe y uma pontuação (score) dada por $\mathbf{w}_y^T \phi(x)$. A partir destes scores, calculamos a probabilidade condicional (posterior) utilizando a transformação softmax:

$$P(y|x) = \frac{\exp(\mathbf{w}_y^T \phi(x))}{Z_x}, \quad Z_x = \sum_{y' \in \mathcal{Y}} \exp(\mathbf{w}_{y'}^T \phi(x)).$$

Onde Z_x é a constante de normalização (partição), definida como a soma das exponenciais de todos os scores possíveis. Apesar de produzir probabilidades, este continua a ser um classificador linear quando selecionamos a classe mais provável. A regra de decisão é:

$$\hat{y} = \arg\max_y P(y|x).$$

Podemos simplificar esta expressão, uma vez que o denominador Z_x é constante para todas as classes e a função exponencial é monótona crescente:

$$\hat{y} = \arg\max_y \exp(\mathbf{w}_y^T \phi(x)) = \arg\max_y \mathbf{w}_y^T \phi(x).$$

As fronteiras de decisão entre duas classes quaisquer, y_i e y_j, são definidas pelo conjunto de pontos onde as suas probabilidades são iguais:

$$P(y_i|x) = P(y_j|x) \implies \mathbf{w}_i^T \phi(x) = \mathbf{w}_j^T \phi(x),$$

$$(\mathbf{w}_i - \mathbf{w}_j)^T \phi(x) = 0.$$

Isto demonstra que as fronteiras de decisão continuam a ser hiperplanos em relação ao espaço das features $\phi(x)$.

Transformação Sigmoid

Para simplificar o modelo multinomial, aproveitamos o facto de que as probabilidades são invariantes se somarmos uma constante a todos os scores. Assim, podemos fixar o score da classe -1 em 0 e o score da classe $+1$ em $\mathbf{w}^T\phi(x)$. A probabilidade da classe positiva é então dada por:

$$P(y = +1|x) = \frac{\exp(\mathbf{w}^T\phi(x))}{1 + \exp(\mathbf{w}^T\phi(x))}.$$

Demonstração: Como vimos anteriormente, num modelo com várias classes, a probabilidade da classe y é:

$$P(y|x) = \frac{\exp(\text{score}_y)}{\sum_j \exp(\text{score}_j)}.$$

Para duas classes ($y = +1$ e $y = -1$), teríamos:

$$P(y = +1|x) = \frac{\exp(\mathbf{w}_{+1}^T\phi(x))}{\exp(\mathbf{w}_{+1}^T\phi(x)) + \exp(\mathbf{w}_{-1}^T\phi(x))}.$$

Repare que, se somarmos um valor constante C a ambos os vetores de pesos ($\mathbf{w}_{+1}$ e $\mathbf{w}_{-1}$), as probabilidades não mudam. Isso significa que o modelo tem parâmetros a mais do que o necessário (está sobre-parametrizado). Para resolver isto, podemos fixar um dos scores como referência. Vamos dividir o numerador e o denominador por $\exp(\mathbf{w}_{-1}^T\phi(x))$:

$$P(y = +1|x) = \frac{\frac{\exp(\mathbf{w}_{+1}^T\phi(x))}{\exp(\mathbf{w}_{-1}^T\phi(x))}}{\frac{\exp(\mathbf{w}_{+1}^T\phi(x))}{\exp(\mathbf{w}_{-1}^T\phi(x))} + \frac{\exp(\mathbf{w}_{-1}^T\phi(x))}{\exp(\mathbf{w}_{-1}^T\phi(x))}}.$$

Utilizando a propriedade das exponenciais $\frac{e^a}{e^b} = e^{a-b}$:

$$P(y = +1|x) = \frac{\exp(\mathbf{w}_{+1}^T\phi(x) - \mathbf{w}_{-1}^T\phi(x))}{\exp(\mathbf{w}_{+1}^T\phi(x) - \mathbf{w}_{-1}^T\phi(x)) + 1}.$$

Agora, definimos um único vetor de pesos $\mathbf{w} = \mathbf{w}_{+1} - \mathbf{w}_{-1}$. Substituindo, chegamos à fórmula:

$$P(y = +1|x) = \frac{\exp(\mathbf{w}^T\phi(x))}{1 + \exp(\mathbf{w}^T\phi(x))}$$

Ao dividirmos o numerador e o denominador por $\exp(\mathbf{w}^T\phi(x))$, obtemos a forma clássica da função sigmóide:

$$P(y=+1|x) = \frac{1}{1+\exp(-\mathbf{w}^T\phi(x))} \equiv \sigma(\mathbf{w}^T\phi(x)).$$

Propriedades da Função Sigmóide $\sigma(a)$, o output está sempre confinado no intervalo $(0,1)$, o que permite interpretá-lo como uma probabilidade. Também, $\sigma(-a) = 1-\sigma(a)$. Isto implica que $P(y=-1|x) = \sigma(-\mathbf{w}^T\phi(x))$. E, por fim, $\sigma(a) = 0.5$ quando $a = 0$. Isto corresponde exatamente ao hiperplano de decisão $\mathbf{w}^T\phi(x) = 0$.

7.3. Multinomial Logistic Regression e Gradient Descent

O modelo é definido pela probabilidade:

$$P_W(y|x) = \frac{\exp(\mathbf{w}_y^T\phi(x))}{Z_x}.$$

Estimativa por Máxima Verosimilhança (MLE). Para aprender os pesos W, maximizamos a log-verosimilhança condicional dos dados de treino:

$$\hat{W} = \arg\max_W \log \prod_{t=1}^{N} P_W(y_t|x_t).$$

Pelas propriedades dos logaritmos, isto equivale a minimizar a log-verosimilhança negativa (Negative Log-Likelihood - NLL):

$$\hat{W} = \arg\min_W -\sum_{t=1}^{N} \log P_W(y_t|x_t).$$

Substituindo a definição de $P_W(y|x)$ na equação acima, obtemos a função de custo que o algoritmo tenta minimizar:

$$\hat{W} = \arg\min_W \sum_{t=1}^{N} L(W;(x_t,y_t))$$

Sendo $L(W;(x,y)) = \log\sum_{y'}\exp(\mathbf{w}_{y'}^T\phi(x)) - \mathbf{w}_y^T\phi(x)$, a função de perda, frequentemente chamada de Cross-Entropy Loss.

Gradient Descent

A regressão logística não tem uma solução algébrica simples para encontrar o W. No entanto, a função de perda é estritamente convexa, o que significa que qualquer mínimo local é também o mínimo global. Para encontrar esse mínimo, usamos métodos iterativos como o Gradient Descent. O objetivo é minimizar uma função diferenciável $f : \mathbb{R}^d \to \mathbb{R}$. A ideia base é seguir a direção de maior descida (o oposto do gradiente):

$$\mathbf{x}^{(t+1)} \leftarrow \mathbf{x}^{(t)} - \eta_t \nabla f(\mathbf{x}^{(t)}).$$

O Step-size/Learning Rate (η_t) é crucial. Se for muito grande, o algoritmo diverge; se for muito pequeno, a convergência é excessivamente lenta. Aplicado ao $W^{(k)}$, temos que:

$$W^{(k+1)} = W^{(k)} - \eta_k \sum_{t=1}^{N} \nabla_W L(W^{(k)};(x_t,y_t)).$$

Começamos com pesos nulos, $W^{(0)} = 0$. Como a função de perda da Regressão Logística é convexa, temos uma garantia de convergência.

7.4. Stochastic Gradient Descent (SGD)

O cálculo do gradiente exato requer a soma dos erros de todos os pontos de dados (Batch Gradient Descent), o que é computacionalmente pesado para grandes conjuntos de dados. A SGD resolve isto aproximando o gradiente usando apenas um único exemplo (ou um pequeno lote, "mini-batch") escolhido aleatoriamente a cada passo. Embora o caminho da SGD seja mais ruidoso e oscilante, é muito mais rápido e garantidamente converge para o ótimo global (com o ajuste adequado do tamanho do passo).

O SGD baseia-se numa aproximação de Monte Carlo do gradiente. Primeiro, inicializamos com $W^{(0)} = 0$. Depois, até à convergência, selecionamos aleatoriamente uma amostra (x_t, y_t) do conjunto de treino e atualizamos os pesos.

$$W^{(k+1)} = W^{(k)} - \eta_k \sum_{t=1}^{N} \nabla_W L(W^{(k)}; (x_t, y_t)).$$

O gradiente de uma única amostra é uma versão "ruidosa" do gradiente real, mas o seu valor esperado é igual ao gradiente total. Uma variante comum, Mini-Batch GD, utiliza um bloco de B amostras (ex.: 32, 64 ou 128) em vez de apenas uma. Isto reduz o ruído do gradiente.

Derivação do Gradiente da Log-Verosimilhança Negativa

Para calcular o gradiente da função de perda $L(W; (x, y))$ em relação à matriz de pesos W, utilizamos as seguintes identidades de cálculo:$\nabla \log F(W) = \frac{\nabla F(W)}{F(W)}$, $\nabla \exp F(W) = \exp(F(W))\nabla F(W)$. Representação One-hot da classe y: $\mathbf{e}_y = [0, \ldots, 0, 1, 0, \ldots, 0]^T$ (onde o 1 está na posição y).

$$\nabla L(W; (x, y)) = \nabla \left(\log \sum_{y'} \exp(\mathbf{w}_{y'}^T \phi(x)) - \mathbf{w}_y^T \phi(x) \right)$$

Primeiro, aplicamos a linearidade do gradiente:

$$= \nabla \log \sum_{y'} \exp(\mathbf{w}_{y'}^T \phi(x)) - \nabla \mathbf{w}_y^T \phi(x)$$

Aplicando a regra da cadeia para o logaritmo e para a exponencial:

$$= \frac{1}{\sum_{y'} \exp(\mathbf{w}_{y'}^T \phi(x))} \sum_{y'} \nabla \exp(\mathbf{w}_{y'}^T \phi(x)) - \mathbf{e}_y \phi(x)^T$$

Substituindo o denominador por Z_x e derivando a exponencial:

$$= \frac{1}{Z_x} \sum_{y'} \exp(\mathbf{w}_{y'}^T \phi(x)) \nabla \mathbf{w}_{y'}^T \phi(x) - \mathbf{e}_y \phi(x)^T$$

Como $\nabla_{\mathbf{w}_{y'}} \mathbf{w}_{y'}^T \phi(x)$ resulta no vetor de características $\phi(x)$ na "linha" correspondente a y', usamos o vetor one-hot $\mathbf{e}_{y'}$ para representar a estrutura matricial:

$$= \sum_{y'} \underbrace{\frac{\exp(\mathbf{w}_{y'}^T \phi(x))}{Z_x}}_{P_W(y'|x)} \mathbf{e}_{y'} \phi(x)^T - \mathbf{e}_y \phi(x)^T$$

Agrupando os termos:

$$= \left(\sum_{y'} P_W(y'|x)\mathbf{e}_{y'} - \mathbf{e}_y \right) \phi(x)^T.$$

Chegamos à forma final elegante, onde o primeiro termo é o vetor de probabilidades previstas:

$$\nabla L(W;(x,y)) = \left(\begin{bmatrix} P_W(1|x) \\ \vdots \\ P_W(|Y||x) \end{bmatrix} - \mathbf{e}_y \right) \phi(x)^T.$$

O gradiente para a linha j da matriz de pesos é:

$$\nabla_{\mathbf{w}_j} L = (P(y=j|x) - \text{target}_j)\phi(x).$$

Se o modelo atribuir $P = 1$ à classe correta e $P = 0$ às restantes, o termo entre parênteses é zero e o gradiente é nulo (os pesos não mudam). O gradiente será proporcional ao erro. Se a classe correta for $y = 1$ mas o modelo prever $P = 0.1$, o termo será $(0.1 - 1) = -0.9$. O sinal negativo no gradiente, quando usado no Gradiente Descendente (que subtrai o gradiente), fará com que os pesos aumentem, tornando a classe correta mais provável na próxima vez.

7.5. Margem

Nem todas as fronteiras de decisão lineares são iguais. Se tivermos duas classes separáveis, podemos desenhar infinitas linhas que as separam. O conceito de margem refere-se à distância entre a fronteira de decisão e os pontos de dados mais próximos. Intuitivamente, e teoricamente, maximizar esta margem é benéfico porque reduz o erro de generalização. Uma margem maior oferece uma "zona de segurança" maior para classificar novos dados não vistos. O Perceptron encontra uma separação, mas não garante a melhor margem; já as Support Vector Machines (SVM) são desenhadas especificamente para maximizar essa margem.

7.6. Regularization

O objetivo passa a ser minimizar não apenas a perda sobre os dados, mas também uma penalização sobre a complexidade dos pesos W:

$$\hat{W} = \arg\min_W \sum_{t=1}^{N} L(W;(x_t,y_t)) + \lambda\Omega(W)$$

Aqui, λ é o hiperparâmetro que controla o compromisso (trade-off) entre ajustar-se bem aos dados (baixo bias) e manter os pesos pequenos (baixa variância).Tipos de Regularização. Regularização ℓ_2 (Ridge), promove pesos globalmente menores, distribuindo a importância por várias características. É definida pela soma dos quadrados dos pesos:

$$\Omega(W) = \sum_y \|\mathbf{w}_y\|_2^2 = \sum_y \sum_i w_{y,i}^2$$

Efeito: Impede que qualquer peso assuma valores extremos. Interpretação: Como vimos anteriormente, equivale a assumir uma prior Gaussiana sobre os pesos. Regularização ℓ_1

(Lasso), promove a esparsidade (coloca muitos pesos exatamente a zero). É definida pela soma dos valores absolutos:

$$\Omega(W) = \sum_y \|\mathbf{w}_y\|_1 = \sum_y \sum_i |w_{y,i}|$$

Efeito: Funciona como uma seleção automática de características (feature selection), eliminando as que não são relevantes para a previsão. Interpretação: Equivale a assumir uma prior de Laplace sobre os pesos.

7.7. Regressão Logística com Regularização ℓ_2

O problema de otimização consiste em minimizar a log-verosimilhança negativa somada a um termo de penalização (Norma de Frobenius ao quadrado, $\|W\|_F^2$, que é a soma de todos os pesos ao quadrado):

$$\min_W \underbrace{-\sum_{t=1}^{N} \log\left(\frac{\exp(\mathbf{w}_{y_t}^T \phi(x_t))}{Z_{x_t}}\right)}_{\sum L(W;(x_t,y_t))} + \frac{\lambda}{2}\|W\|_F^2$$

O Cálculo do Gradiente. Para aplicar o Gradiente Descendente, derivamos a função total. Sabemos que:

$$\nabla_W \|W\|_F^2 = 2W$$

Portanto, o gradiente total é:

$$\nabla_{\text{Total}} = \sum_{t=1}^{N} \nabla L(W; (x_t, y_t)) + \lambda W$$

Passo de Atualização: O conceito de "Weight Decay". Ao substituir o gradiente na regra de atualização do Gradiente Descendente, obtemos uma forma muito interessante:

$$W^{(k+1)} = W^{(k)} - \eta_k \left(\sum_{t=1}^{N} \nabla L(W^{(k)}; (x_t, y_t)) + \lambda W^{(k)}\right)$$

Reorganizando os termos que dependem de $W^{(k)}$:

$$W^{(k+1)} = \underbrace{(1 - \eta_k \lambda)}_{\text{Weight Decay}} W^{(k)} - \eta_k \sum_{t=1}^{N} \nabla L(W^{(k)}; (x_t, y_t))$$

7.8. Caminho para Deep Learning

Apesar destas técnicas, os classificadores lineares têm uma limitação fundamental: não conseguem resolver problemas que não são linearmente separáveis, como o clássico problema XOR. Para resolver isto, temos três opções principais.

Engenharia de Atributos: Transformar manualmente os dados para um espaço onde sejam separáveis.

Métodos de Kernel: Mapear implicitamente para dimensões superiores. Redes Neuronais: Em vez de desenhar atributos à mão, desenhamos uma arquitetura (camadas de transformações não-lineares) que aprende a transformar os dados numa representação linearmente separável.

8. Lecture 4 and 5: Neural Networks I & II

8.1. O Neurónio Artificial

A unidade fundamental de uma rede neuronal processa a informação em dois passos:

- **Pré-ativação:** Corresponde à soma ponderada das entradas x_i pelos pesos w_i, adicionando-se o bias b:

$$z(x) = \mathbf{w}^T\mathbf{x} + b = \sum_{i=1}^{D} w_i x_i + b \tag{2}$$

- **Ativação:** Aplicação de uma função não-linear g para obter a saída do neurónio $h(x)$:

$$h(x) = g(z(x)) = g(\mathbf{w}^T\mathbf{x} + b) \tag{3}$$

8.2. Activation Functions

A escolha da função não-linear é crucial. Historicamente, usava-se a Sigmoid ($\sigma(z)$) ou a Tangente Hiperbólica ($\tanh(z)$). Embora tenham interpretações claras (probabilidade ou sinal), elas sofrem do problema de saturação: para valores muito altos ou baixos, a curva torna-se plana, o gradiente aproxima-se de zero e a aprendizagem para ("Vanishing Gradient"). A solução moderna padrão é a ReLU.

Função	**Definição Matemática**	**Intervalo**	**Principais Propriedades**
Linear	$f(z) = z$	$(-\infty, +\infty)$	Sem compressão; camadas múltiplas equivalem a uma.
Sigmóide	$\sigma(z) = \frac{1}{1+e^{-z}}$	$(0, 1)$	Interpretável como probabilidade; estritamente crescente.
Tanh	$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$(-1, 1)$	Centrada no zero; estritamente crescente; limitada (*bounded*).
ReLU	$\max(0, z)$	$[0, +\infty)$	Esparsidade; mitiga o *vanishing gradient*; inspiração biológica.

Table 1: Comparação de Funções de Ativação.

8.3. Resolução do XOR com Multi-Layer Perceptron

O problema XOR serviu para demonstrar a necessidade de arquiteturas multicamada.

- **Conectividade:** Segundo o teorema de Minsky e Papert (1969), para implementar funções lógicas arbitrárias, as unidades escondidas devem estar conectadas a todas as entradas.

- **Complexidade:** Isto contrariou a esperança inicial de que neurónios com conectividade apenas local seriam suficientes para resolver problemas de lógica complexa.

8.4. Multilayer Neural Networks

As redes neuronais modernas baseiam-se na composição de múltiplas camadas de processamento.

- **Camadas Escondidas:** A utilização de camadas intermédias permite que a rede aprenda hierarquias de características (representations).
- **Não-linearidade:** Ao combinar múltiplas camadas com funções de ativação não-lineares, o poder expressivo da rede aumenta, permitindo modelar fronteiras de decisão complexas que não seriam possíveis com modelos lineares simples.
- **Fluxo de Informação:** Nestas redes, designadas por *feed-forward*, a informação é propagada exclusivamente da entrada para a saída, sem ciclos ou retroalimentação.

8.5. Single Hidden Layer, Single Output

Consideramos um modelo com D entradas ($\mathbf{x} \in \mathbb{R}^D$), uma camada intermédia (escondida) com K unidades ($\mathbf{h} \in \mathbb{R}^K$) e uma única saída. A pré-ativação da camada escondida é uma transformação linear dos inputs:

$$\mathbf{z}(\mathbf{x}) = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$$

Onde $\mathbf{W}^{(1)} \in \mathbb{R}^{K \times D}$ é a matriz de pesos e $\mathbf{b}^{(1)} \in \mathbb{R}^K$ é o vetor de bias. A ativação da camada escondida é obtida aplicando uma função g elemento a elemento:

$$\mathbf{h}(\mathbf{x}) = g(\mathbf{z}(\mathbf{x}))$$

Camada de Saída. A saída final do modelo, $f(\mathbf{x})$, é calculada projetando a representação escondida $\mathbf{h}(\mathbf{x})$ para o espaço de saída:

$$f(\mathbf{x}) = o(\mathbf{h}(\mathbf{x})^T\mathbf{w}^{(2)} + b^{(2)})$$

Onde $\mathbf{w}^{(2)} \in \mathbb{R}^K$ e o é a função de ativação de saída. A função completa da rede pode ser escrita como uma composição:

$$f(\mathbf{x}) = o\left(\mathbf{w}^{(2)T} g(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) + b^{(2)}\right)$$

Dependendo do problema, a função de saída o varia: Regressão ($y \in \mathbb{R}$): $o(u) = u$ (identidade). Classificação Binária ($y \in \{\pm 1\}$): $o(u) = \sigma(u)$, onde $f(\mathbf{x}) = P(y = 1|x)$.

8.6. Multiple Classes

Na classificação multiclasse, o objetivo é mapear o input para uma de C categorias possíveis. Para tal, a rede neuronal é configurada com C unidades de saída, onde cada unidade c estima a probabilidade $P(y = c|\mathbf{x})$. A classe prevista é, geralmente, aquela que apresenta a maior probabilidade estimada. As saídas da rede devem residir no simplex de probabilidade $\Delta^{C-1} \subseteq \mathbb{R}^C$, definido como:

$$\Delta^{C-1} = \left\{ (p_1, \ldots, p_C) \in \mathbb{R}^C : p_i \geq 0, \sum_{i=1}^{C} p_i = 1 \right\}$$

A função de ativação ideal para garantir que as saídas formem uma distribuição de probabilidade válida é a softmax. Esta função mapeia um vetor de scores $z \in \mathbb{R}^C$ para o simplex:

$$\mathbf{o}(\mathbf{z}) = \text{softmax}(\mathbf{z}) = \left[\frac{\exp(z_1)}{\sum_{c=1}^{C} \exp(z_c)}, \ldots, \frac{\exp(z_C)}{\sum_{c=1}^{C} \exp(z_c)} \right]$$

Propriedades Fundamentais: Esta é a generalização da sigmóide para múltiplas classes. Todos os valores resultantes são estritamente positivos. A soma de todas as componentes é garantidamente igual a 1. A distribuição resultante tem suporte total em todas as classes.

8.7. Multiple ($L \geq 1$) Hidden Layers

Em redes profundas, a saída de cada camada serve como entrada para a camada seguinte, criando uma hierarquia de representações. Para cada camada ℓ (onde $\ell = 1, \ldots, L$), definimos a pré-ativação como uma transformação linear da ativação da camada anterior:

$$\mathbf{z}^{(\ell)}(\mathbf{x}) = \mathbf{W}^{(\ell)}\mathbf{h}^{(\ell-1)}(\mathbf{x}) + \mathbf{b}^{(\ell)}$$

Define-se $\mathbf{h}^{(0)} = \mathbf{x}$ (a entrada da rede) para manter a consistência da fórmula. $\mathbf{W}^{(\ell)} \in \mathbb{R}^{K_\ell \times K_{\ell-1}}$ e $\mathbf{b}^{(\ell)} \in \mathbb{R}^{K_\ell}$, onde K_ℓ é o número de unidades na camada ℓ. A ativação de cada camada é obtida aplicando a função de ativação não-linear g:

$$\mathbf{h}^{(\ell)}(\mathbf{x}) = g\left(\mathbf{z}^{(\ell)}(\mathbf{x})\right)$$

A ativação final da rede, após passar por todas as L camadas escondidas, é calculada na camada $L + 1$:

$$\mathbf{f}(\mathbf{x}) = o\left(\mathbf{z}^{(L+1)}(\mathbf{x})\right) = o\left(\mathbf{W}^{(L+1)}\mathbf{h}^{(L)}(\mathbf{x}) + \mathbf{b}^{(L+1)}\right)$$

Aqui, o representa a função de ativação de saída (ex: Softmax para classificação multi-classe ou Identidade para regressão).

8.8. Universal Approximation Theorem

O Teorema da Aproximação Universal afirma que uma rede feedforward com apenas uma camada escondida (e não-linearidades adequadas) consegue aproximar qualquer função contínua com a precisão que desejarmos, desde que a camada escondida tenha neurónios suficientes. No entanto, este teorema é uma garantia teórica de existência, não de aprendizagem. Na prática, redes mais profundas (com mais camadas) são geralmente mais eficientes em termos de parâmetros e generalização do que redes extremamente largas e rasas.

Deeper Networks

Pelo Teorema (Montufar) O número de regiões lineares que podem ser geradas por uma rede neuronal profunda com D entradas, profundidade L e K unidades escondidas por camada (utilizando ativações ReLU) é dado por:

$$O\left(\left(\frac{K}{D}\right)^{D(L-1)} K^D\right)$$

Para um número fixo de neurónios por camada (K), o número de regiões lineares cresce exponencialmente com a profundidade L. Redes profundas conseguem representar funções

altamente complexas com muito menos parâmetros do que seria necessário numa rede de apenas uma camada escondida. Enquanto uma rede rasa precisaria de um número exponencial de neurónios para criar o mesmo número de partições, uma rede profunda consegue-o através da composição de funções simples camada após camada.

8.9. Empirical Risk Minimization (ERM)

Embora as redes neuronais sejam modelos altamente expressivos — capazes de aproximar qualquer função em teoria — o seu desempenho depende da escolha correta do conjunto de parâmetros:

$$\theta := \{(\mathbf{W}^{(1)}, \mathbf{b}^{(1)}), \ldots, (\mathbf{W}^{(L+1)}, \mathbf{b}^{(L+1)})\}$$

A ideia central é aprender estes parâmetros a partir de dados, amostrando pontos e os seus respetivos valores. O objetivo é encontrar os parâmetros θ que minimizam uma função de custo global, composta pela perda média nos dados de treino e um termo de regularização:

$$\mathcal{L}(\theta) := \lambda\Omega(\theta) + \frac{1}{N}\sum_{i=1}^{N} L(f(\mathbf{x}_i;\theta), y_i)$$

Os $\{(\mathbf{x}_i, y_i), i = 1, \ldots, N\}$ representam os dados observados. O L é a Loss Function. O $\Omega(\theta)$ é o regularizador. E o λ é a constante de regularização (hyperparameter to be tuned).

SGD Recap

Podemos reescrever a função de custo global como a média das perdas individuais por amostra, facilitando a análise do gradiente:

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \underbrace{(\lambda\Omega(\theta) + L(f(\mathbf{x}_i;\theta), y_i))}_{\mathcal{L}_i(\theta)} = \frac{1}{N}\sum_{i=1}^{N} \mathcal{L}_i(\theta)$$

O gradiente exato da função de custo é a média dos gradientes de todas as amostras:

$$\nabla_\theta \mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \nabla_\theta \mathcal{L}_i(\theta)$$

Limitação do Batch GD: Requer uma passagem completa por todo o conjunto de dados para uma única atualização, o que é proibitivamente lento em grandes datasets. Aproximação Estocástica: Ao amostrar aleatoriamente uma única amostra j, obtemos um estimador não enviesado do gradiente, mas com elevada variância (ruído).

SGD com Mini-Batches

Na prática, utiliza-se um compromisso entre os dois extremos: o Mini-batch SGD. Em vez de uma ou todas as amostras, selecionamos um subconjunto $B \ll N$. O gradiente é aproximado pela média dos gradientes das amostras no mini-batch $\{j_1, \ldots, j_B\}$:

$$\nabla_\theta \mathcal{L}(\theta) \approx \lambda\nabla_\theta\Omega(\theta) + \frac{1}{B}\sum_{i=1}^{B} \nabla_\theta L(f(\mathbf{x}_{j_i};\theta), y_{j_i})$$

Os pesos e biases da rede, $\theta = \{(\mathbf{W}^{(\ell)}, \mathbf{b}^{(\ell)})\}_{\ell=1}^{L+1}$, são atualizados subtraindo o gradiente escalado pela taxa de aprendizagem η:

$$\theta \leftarrow \theta - \eta \left(\frac{1}{B} \sum_{i=1}^{B} \nabla_\theta \mathcal{L}_{j_i}(\theta) \right)$$

8.10. Key Ingredients of SGD

De forma resumida, a implementação do SGD exige três componentes principais: Função de Perda $L(f(x_i;\theta), y_i)$, Procedimento de Cálculo de Gradientes $\nabla_\theta L(f(x_i;\theta), y_i)$, e, por fim, Regularizador $\Omega(\theta)$ e o seu Gradiente.

Loss Function

A Squared Loss é a escolha padrão para problemas de regressão ou reconstrução, onde o objetivo é que a saída da rede $f(\mathbf{x};\theta)$ seja o mais próxima possível do valor real y.

$$L(f(\mathbf{x};\theta), y) = \frac{1}{2}\|f(\mathbf{x};\theta) - y\|^2$$

O gradiente em relação a cada componente da saída f_c é simplesmente a diferença entre a previsão e o alvo:

$$\frac{\partial L}{\partial f_c} = f_c(\mathbf{x};\theta) - y_c$$

A Entropia Cruzada (Negative Log-Likelihood) é a escolha comum quando a camada de saída utiliza uma ativação softmax, sendo ideal para classificação multiclasse. Aqui, a rede estima a probabilidade de cada classe c:

$$L(f(\mathbf{x};\theta), y) = -\sum_c \mathbf{1}(y = c) \log f_c(\mathbf{x};\theta) = -\log f_y(\mathbf{x};\theta)$$

Onde $f_y(\mathbf{x};\theta)$ é a probabilidade que a rede atribui à classe correta y. Uma propriedade notável da combinação Softmax + Entropia Cruzada é a simplicidade do seu gradiente em relação aos scores brutos (z):

$$\frac{\partial L}{\partial z_c} = \text{softmax}(\mathbf{z}(\mathbf{x}))_c - \mathbf{1}(y = c)$$

Gradient Computation

Se uma função $r(t)$ depende de resultados intermédios $q_i(t)$, a sua derivada em relação a t é a soma das derivadas parciais através de todos os caminhos possíveis:

$$\frac{\partial r(t)}{\partial t} = \sum_i \frac{\partial r(t)}{\partial q_i(t)} \frac{\partial q_i(t)}{\partial t}$$

Na arquitetura de uma rede neuronal, $r(t)$ é a função de perda (L), $q_i(t)$ representa a pré-ativação na camada superior, e t é um parâmetro (peso ou bias) ou uma unidade de saída numa camada inferior. A mensagem principal é que o backpropagation é a aplicação da regra da cadeia. O gradiente em relação à ativação da camada ℓ ($\mathbf{h}^{(\ell)}$) é calculado acumulando os erros da camada superior ($\ell + 1$) através da regra da cadeia:

$$\frac{\partial L(f(x;\theta),y)}{\partial h_j^{(\ell)}} = \sum_i \frac{\partial L}{\partial z_i^{(\ell+1)}} \frac{\partial z_i^{(\ell+1)}}{\partial h_j^{(\ell)}} = \sum_i \frac{\partial L}{\partial z_i^{(\ell+1)}} W_{i,j}^{(\ell+1)}$$

Em notação vetorial:

$$\nabla_{\mathbf{h}^{(\ell)}} L = \mathbf{W}^{(\ell+1)T} \nabla_{\mathbf{z}^{(\ell+1)}} L$$

Para passar da ativação $\mathbf{h}^{(\ell)}$ para a pré-ativação $\mathbf{z}^{(\ell)}$, aplicamos a derivada da função de ativação g:

$$\frac{\partial L(f(x;\theta),y)}{\partial z_j^{(\ell)}} = \frac{\partial L}{\partial h_j^{(\ell)}} \frac{\partial h_j^{(\ell)}}{\partial z_j^{(\ell)}} = \frac{\partial L}{\partial h_j^{(\ell)}} g'(z_j^{(\ell)})$$

Em notação vetorial, usamos o produto de Hadamard (elemento a elemento, $\odot$):

$$\nabla_{\mathbf{z}^{(\ell)}} L = \nabla_{\mathbf{h}^{(\ell)}} L \odot g'(\mathbf{z}^{(\ell)})$$

Para completar o cálculo, precisamos das derivadas das funções que tabelámos anteriormente. Uma vez calculado o erro na pré-ativação de uma camada ℓ, definido como $\nabla_{\mathbf{z}^{(\ell)}} L$ (ou frequentemente denotado por $\delta^{(\ell)}$), podemos determinar como cada parâmetro individual contribui para esse erro utilizando a regra da cadeia. O peso $W_{i,j}^{(\ell)}$ liga o neurónio j da camada anterior $(\ell-1)$ ao neurónio i da camada atual (ℓ). A sua contribuição para a perda é o produto do erro acumulado "à frente" pelo sinal que veio "de trás":

$$\frac{\partial L}{\partial W_{i,j}^{(\ell)}} = \frac{\partial L}{\partial z_i^{(\ell)}} \frac{\partial z_i^{(\ell)}}{\partial W_{i,j}^{(\ell)}} = \frac{\partial L}{\partial z_i^{(\ell)}} h_j^{(\ell-1)}$$

Em notação matricial, isto resulta num produto externo:

$$\nabla_{\mathbf{W}^{(\ell)}} L = (\nabla_{\mathbf{z}^{(\ell)}} L)(\mathbf{h}^{(\ell-1)})^T$$

Como o bias $b_i^{(\ell)}$ tem uma derivada de 1 em relação à pré-ativação $z_i^{(\ell)}$, o seu gradiente é simplesmente o próprio sinal de erro da camada:

$$\frac{\partial L}{\partial b_i^{(\ell)}} = \frac{\partial L}{\partial z_i^{(\ell)}} \frac{\partial z_i^{(\ell)}}{\partial b_i^{(\ell)}} = \frac{\partial L}{\partial z_i^{(\ell)}} \cdot 1$$

Em notação vetorial:

$$\nabla_{\mathbf{b}^{(\ell)}} L = \nabla_{\mathbf{z}^{(\ell)}} L$$

Resumo do Algoritmo Backpropagation

Podemos agora sistematizar o fluxo completo para uma amostra (x, y). 1) Forward Pass: Calcula as ativações $\mathbf{h}^{(\ell)}$ de todas as camadas até à saída $f(\mathbf{x})$. 2) Output Error: Calcula o erro inicial na camada de saída, $\nabla_{\mathbf{z}^{(L+1)}} L$ (ex: $\hat{y} - y$ para Softmax). 3) Backward Pass: Para cada camada ℓ de L até 1: Calcula os gradientes dos parâmetros $\nabla_{\mathbf{W}^{(\ell+1)}} L$ e $\nabla_{\mathbf{b}^{(\ell+1)}} L$ usando os resultados acima. Propaga o erro para a camada anterior: $\nabla_{\mathbf{z}^{(\ell)}} L = (\mathbf{W}^{(\ell+1)T} \nabla_{\mathbf{z}^{(\ell+1)}} L) \odot g'(\mathbf{z}^{(\ell)})$.

Regularization

Para além das técnicas de regularização estrutural como ℓ_1 e ℓ_2, o Dropout introduz uma abordagem estocástica para combater o overfitting. O Dropout não segue a forma tradicional de adicionar uma penalização à função de custo. Em vez disso, durante o treino, o algoritmo remove aleatoriamente unidades escondidas da rede.

- **Procedimento:** Cada unidade da camada escondida é desativada com probabilidade p, utilizando uma máscara binária $\mathbf{m}^{(\ell)}$:

$$\mathbf{h}^{(\ell)} = g(\mathbf{z}^{(\ell)}) \odot \mathbf{m}^{(\ell)} \tag{4}$$

- **Inverted Dropout:** Para manter a escala das ativações, os valores não omitidos são reescalados por $1/(1-p)$ durante o treino, facilitando a fase de inferência.

- **Vantagem:** Reduz a co-adaptação entre neurónios, forçando a rede a aprender representações mais redundantes e robustas.

Compute output gradient (before activation):

$$\nabla_{\boldsymbol{z}^{(L+1)}} L(\boldsymbol{f}(\boldsymbol{x};\boldsymbol{\theta}), y) = -(1_y - \boldsymbol{f}(\boldsymbol{x}))$$

for ℓ from $L+1$ to 1 **do**

Compute gradients of hidden layer parameters:

$$\nabla_{\boldsymbol{W}^{(\ell)}} L(\boldsymbol{f}(\boldsymbol{x};\boldsymbol{\theta}), y) = \nabla_{\boldsymbol{z}^{(\ell)}} L(\boldsymbol{f}(\boldsymbol{x};\boldsymbol{\theta}), y) \underbrace{\boldsymbol{h}^{(\ell-1)\top}}_{\text{includes } \boldsymbol{m}^{(\ell-1)}}$$

$$\nabla_{\boldsymbol{b}^{(\ell)}} L(\boldsymbol{f}(\boldsymbol{x};\boldsymbol{\theta}), y) = \nabla_{\boldsymbol{z}^{(\ell)}} L(\boldsymbol{f}(\boldsymbol{x};\boldsymbol{\theta}), y)$$

Compute gradient of hidden layer below:

$$\nabla_{\boldsymbol{h}^{(\ell-1)}} L(\boldsymbol{f}(\boldsymbol{x};\boldsymbol{\theta}), y) = \boldsymbol{W}^{(\ell)\top} \nabla_{\boldsymbol{z}^{(\ell)}} L(\boldsymbol{f}(\boldsymbol{x};\boldsymbol{\theta}), y)$$

Compute gradient of hidden layer below (before activation):

$$\nabla_{\boldsymbol{z}^{(\ell-1)}} L(\boldsymbol{f}(\boldsymbol{x};\boldsymbol{\theta}), y) = \nabla_{\boldsymbol{h}^{(\ell-1)}} L(\boldsymbol{f}(\boldsymbol{x};\boldsymbol{\theta}), y) \odot \boldsymbol{g}'(\boldsymbol{z}^{(\ell-1)}) \odot \boldsymbol{m}^{(\ell-1)}$$

end for

Figure 2: Backpropagation with Dropout.

8.11. Initialization

A convergência de uma rede neuronal profunda é altamente dependente da inicialização dos seus pesos e biases.

- **Biases:** São geralmente inicializados a zero.

- **Pesos:** Devem ser inicializados aleatoriamente para quebrar a simetria. Uma inicialização idêntica impediria os neurónios de aprenderem diferentes representações.

- **Glorot Initialization:** Para ativações *tanh* ou *sigmoid*, recomenda-se a inicialização de Glorot:

$$W_{i,j}^{(\ell)} \sim U\left[-\frac{\sqrt{6}}{\sqrt{K^{(\ell)} + K^{(\ell-1)}}}, \frac{\sqrt{6}}{\sqrt{K^{(\ell)} + K^{(\ell-1)}}}\right]$$

8.12. Metodologias de Treino e Avaliação

Divisão de Dados e Generalização

Para garantir que o modelo generaliza para dados não vistos, o dataset deve ser dividido em três partições distintas: Treino (Training set), utilizado para atualizar os parâmetros $(\mathbf{W}, \mathbf{b})$ do modelo. alidação (Validation set), utilizado para ajustar hiperparâmetros (taxa de aprendizagem, número de neurónios, λ, probabilidade de dropout, etc.). Teste (Test set), utilizado exclusivamente para estimar o desempenho final de generalização.

Otimização de Hiperparâmetros

Grid Search: Testa-se exaustivamente todas as combinações de um conjunto predefinido de valores. Random Search: Amostra-se configurações de forma independente a partir de distribuições estatísticas; muitas vezes mais eficiente que o Grid Search. Otimização Bayesiana: Abordagem mais avançada que utiliza modelos probabilísticos para guiar a procura.

Early Stopping

O Early Stopping consiste em interromper o treino no momento em que o erro de validação começa a aumentar (mesmo que o erro de treino continue a diminuir). Uma técnica comum em SGD é reduzir a taxa de aprendizagem quando o erro de validação sobe e parar o treino após alguns ciclos sem melhoria.

Estabilização do Treino: Normalização e Decaimento

Normalização de Dados: Para cada dimensão do input, subtrai-se a média e divide-se pelo desvio padrão do conjunto de treino. Isto garante média zero e variância unitária, o que acelera significativamente a convergência. No entanto, não deve ser usada em inputs esparsos, pois destrói a esparsidade.

Decaimento da Taxa de Aprendizagem (Learning Rate Decay): À medida que o SGD se aproxima de um mínimo local, convém reduzir o passo de atualização para evitar divergência. Uma estratégia comum é dividir a taxa por 2 sempre que o erro de validação pare de melhorar.

Eficiência com Mini-Batches

Em vez de atualizar os pesos após cada exemplo individual, agregamos um subconjunto de exemplos (tipicamente entre 50 e 200) para calcular um gradiente médio. O gradiente resultante é menos ruidoso do que no SGD padrão. A computação é trivialmente paralelizável entre o mini-batch, permitindo esgotar a memória da GPU para ganhos massivos de velocidade.

Gradient Checking

Se a perda de treino não diminuir mesmo com uma taxa de aprendizagem muito pequena, é provável que exista um erro no cálculo do gradiente. A depuração do forward e backward prop é feita comparando o gradiente analítico com o gradiente numérico:

$$\frac{\partial f(x)}{\partial x} \approx \frac{f(x+\epsilon) - f(x-\epsilon)}{2\epsilon}$$

Depuração com Conjuntos de Dados Pequenos

Uma técnica eficaz para validar o modelo consiste em tentar "decorar" um subconjunto reduzido de dados (ex: 50 exemplos). Objetivo: deve ser capaz de atingir um overfit total neste conjunto pequeno. Diagnóstico de Saturação: se não conseguir, verifique se existem unidades saturadas desde as primeiras iterações (neste caso, reduza a variância da inicialização ou normalize melhor os inputs). Estabilidade: se o erro de treino oscilar bruscamente para cima e para baixo, diminua a taxa de aprendizagem.

Momentum

O Momentum ajuda a navegar em superfícies de erro onde o gradiente é muito mais inclinado numa direção do que noutras, acumulando velocidade nas direções estáveis. Regra de Atualização: $\theta_t = \theta_{t-1} - \alpha_t g(\theta_{t-1}) + \gamma_t(\theta_{t-1} - \theta_{t-2})$.Reduz oscilações em direções de gradiente variável e aumenta o passo em direções de gradiente constante.

AdaGrad (Adaptive Gradient)

Ajusta as taxas de aprendizagem individualmente para cada parâmetro θ_j com base no histórico dos gradientes. Mecanismo: $\theta_{j,t} = \theta_{j,t-1} - \frac{\alpha}{\sqrt{G_{j,t-1}+\epsilon}} g_j(\theta_{t-1})$, onde $G_{j,t}$ é a soma dos quadrados dos gradientes passados. Robusto à escolha da taxa de aprendizagem inicial e a diferentes escalas de parâmetros. A taxa de aprendizagem pode tornar-se excessivamente pequena (vanishing step size) porque $G_{j,t}$ cresce monotonamente.

RMSProp (Root Mean Square Propagation)

Resolve o problema do decréscimo excessivo da taxa de aprendizagem no AdaGrad através de um fator de esquecimento. Mecanismo: Utiliza uma média móvel exponencial dos quadrados dos gradientes: $G_{j,t} = \gamma G_{j,t-1} + (1-\gamma)(g_j(\theta_t))^2$. Mantém a robustez à escala dos parâmetros, permitindo que a taxa de aprendizagem recupere se necessário.

Adam (Adaptive Moment Estimation)

Combina as vantagens do RMSProp e do Momentum, sendo atualmente um dos otimizadores mais utilizados. Mantém médias móveis para o gradiente (m_t) e para o quadrado do gradiente (v_t), com correções de bias para as iterações iniciais: $\hat{m}_t = m_t/(1-\beta_1^t)$ e $\hat{v}_t = v_t/(1-\beta_2^t)$. Atualização: $\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\epsilon}$. Eficiente computacionalmente e com baixo consumo de memória, ideal para grandes volumes de dados.

9. Lecture 6: Representation Learning

9.1. Representações e Aprendizagem Hierárquica

As Redes Neuronais (NNs) aprendem representações $\phi(\mathbf{x})$ que tornam os modelos mais expressivos e precisos, permitindo inclusive a transferência dessas representações entre diferentes tarefas. As camadas mais próximas dos inputs aprendem representações de baixo nível, como cantos e arestas. À medida que avançamos na rede, as camadas

aprendem conceitos mais abstratos, como formas, moldes e objetos complexos. Este padrão de aprendizagem hierárquica aplica-se não apenas a imagens, mas também a texto, sons e outros tipos de dados.

Um conceito central introduzido por Hinton (1984) é a distinção entre como a informação é codificada na rede. Local (one-hot): Cada dimensão/neurónio corresponde a um objeto único (Um neurónio específico para "Gato", outro para "Cão"), e Distribuída: Cada dimensão representa uma propriedade; o objeto é definido pela combinação de várias ativações (Neurónios para "Peludo", "Quatro patas", "Orelhas pontiagudas"). No single neuron "encodes" any concept; groups of neurons (e.g. in the same hidden layer) work together!

9.2. Power of Distributed Representations

As representações distribuídas são muito mais compactas que as locais; com N propriedades, é possível representar $O(e^N)$ objetos diferentes através das suas combinações. Estas representações permitem que o modelo generalize para objetos nunca antes vistos de uma forma significativa, baseando-se na sobreposição de propriedades conhecidas.

Para que estas representações sejam eficazes, as unidades escondidas devem capturar propriedades diversas dos objetos. É fundamental que as unidades não capturem todas a mesma propriedade. Isto é garantido através da inicialização aleatória dos pesos. Se todas as unidades forem inicializadas com os mesmos pesos, a simetria nunca seria quebrada, resultando em unidades que aprendem exatamente o mesmo.

9.3. Unsupervised Pre-Training

O treino de redes neuronais profundas com muitas camadas era extremamente difícil devido a problemas de convergência e gradientes. Uma solução eficaz é inicializar as camadas escondidas utilizando aprendizagem não supervisionada. O objetivo é forçar a rede a representar a estrutura latente da distribuição dos inputs. Incentiva-se as camadas escondidas a codificar essa estrutura intrínseca antes de se iniciar o treino supervisionado. Este processo pode ser realizado eficientemente através de um Auto-encoder.

Data Manifold

A ideia central por trás desta abordagem é que os objetos de entrada não estão distribuídos aleatoriamente no espaço, mas sim numa variedade (manifold) de dimensão inferior. O modelo tenta aprender a superfície ou o espaço onde os objetos "vivem".

9.4. Auto-Encoders

Um Auto-encoder é uma rede feed-forward treinada especificamente para reproduzir o seu input na saída. Temos:

- Encoder: Mapeia o input para uma representação escondida: $\mathbf{h}(\mathbf{x}) = g(\mathbf{W}\mathbf{x} + \mathbf{b})$.
- Decoder: Tenta reconstruir o input original a partir de $\mathbf{h}$: $\hat{\mathbf{x}} = \mathbf{W}^T\mathbf{h}(\mathbf{x}) + \mathbf{c}$.
- Função de Perda: Para dados em $\mathbb{R}^D$, utiliza-se geralmente a norma quadrática: $L(\hat{\mathbf{x}}; \mathbf{x}) = \|\hat{\mathbf{x}} - \mathbf{x}\|^2$.

Otimizamos os pesos para minimizar o erro de reconstrução em todo o dataset (omitindo os biases para simplificação):

$$\mathbf{W}^* = \arg\min_{\mathbf{W}} \sum_i \|\mathbf{W}^T g(\mathbf{W}\mathbf{x}_i) - \mathbf{x}_i\|^2$$

9.5. Singular Value Decomposition (SVD)

A SVD estabelece que qualquer matriz $\mathbf{A} \in \mathbb{R}^{m\times n}$ com característica (rank) r pode ser fatorada como o produto de três matrizes específicas:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$$

$\mathbf{U} \in \mathbb{R}^{m\times r}$: As suas colunas formam uma base ortonormal para o espaço das colunas de $\mathbf{A}$, satisfazendo $\mathbf{U}^T\mathbf{U} = \mathbf{I}$. $\mathbf{V} \in \mathbb{R}^{n\times r}$: As suas colunas formam uma base ortonormal para o espaço das linhas de $\mathbf{A}$, satisfazendo $\mathbf{V}^T\mathbf{V} = \mathbf{I}$. $\mathbf{\Sigma} \in \mathbb{R}^{r\times r}$: Uma matriz diagonal $\text{diag}(\sigma_1, \ldots, \sigma_r)$ contendo os valores singulares.

Os valores singulares são as raízes quadradas dos valores próprios (eigenvalues) de $\mathbf{A}^T\mathbf{A}$ ou $\mathbf{A}\mathbf{A}^T$. Representam a "força" ou importância de cada dimensão na reconstrução da matriz original.

9.6. Linear Auto-Encoder e PCA

Quando utilizamos um Auto-encoder sem funções de ativação não-lineares e com dados centrados, o objetivo de minimizar o erro de reconstrução equivale a encontrar uma aproximação de baixo rank para a matriz de dados. Seja $\mathbf{X} \in \mathbb{R}^{N\times D}$ a matriz de dados e $\mathbf{W} \in \mathbb{R}^{K\times D}$ a matriz de pesos com $K < D$. Queremos minimizar a norma de Frobenius do erro de reconstrução:

$$\|\mathbf{X} - \hat{\mathbf{X}}\|_F^2 = \|\mathbf{X} - \mathbf{X}\mathbf{W}^T\mathbf{W}\|_F^2$$

Pelo Teorema de Eckart-Young, a solução ótima é obtida através da SVD truncada de $\mathbf{X}^T$. A solução é dada por $\mathbf{W} = \mathbf{U}_K^T$, onde $\mathbf{U}_K$ contém os K principais vetores singulares esquerdos de $\mathbf{X}^T$. Este procedimento é exatamente a Análise de Componentes Principais (PCA). PCA fits a linear manifold.

9.7. Auto-Encoder, Regularização e Variantes

Ao introduzir funções de ativação não-lineares (g), o Auto-encoder deixa de estar limitado a variedades lineares, permitindo obter códigos (representações) muito mais sofisticados. No entanto, é necessária regularização para promover representações suaves e para evitar o overfitting. Existem várias arquiteturas desenhadas para forçar a rede a aprender características úteis:

- Sparse Auto-encoders: Utilizam muitas unidades escondidas, mas aplicam uma regularização ℓ_1 para incentivar representações esparsas.
- Denoising Auto-encoders: O modelo aprende a reconstruir o input original a partir de uma versão corrompida por ruído, forçando a aprendizagem de uma função de mapeamento robusta e suave.
- Stacked Auto-encoders: Composição de vários auto-encoders uns sobre os outros para profundidade acrescida.
- Variational Auto-encoders (VAE): Um modelo probabilístico generativo que minimiza um limite variacional (tópico avançado).

A regularização é fundamental para garantir que o Auto-encoder aprenda características estruturais úteis em vez de apenas memorizar os dados de treino. O objetivo passa a ser a minimização de uma função de custo composta pela perda de reconstrução e um termo de penalização Ω:

$$\mathcal{L}(\hat{\mathbf{x}}; \mathbf{x}) + \Omega(\mathbf{h}, \mathbf{x})$$

Exemplos comuns de regularização incluem a penalização direta do código ($\Omega = \lambda \|\mathbf{h}\|^2$) ou a regularização das derivadas para garantir que pequenas mudanças no input não alterem drasticamente a representação ($\Omega = \lambda \sum_i \|\nabla_{\mathbf{x}} h_i\|^2$).

Sparse Auto-encoders

Os Auto-encoders esparsos podem ter um número de unidades escondidas superior à dimensão do input ($K > D$). Adiciona-se um termo $\Omega(\mathbf{h}) = \lambda \|\mathbf{h}\|_1$. Incentivar a rede a manter a maioria dos neurónios inativos para qualquer entrada específica, forçando a seleção apenas das características mais relevantes.

Denoising Auto-encoders

Esta variante utiliza uma forma de regularização implícita para garantir a suavidade da representação. O input é corrompido com ruído aleatório $\tilde{\mathbf{x}} = \mathbf{x} + \mathbf{n}$ (ex: ruído Gaussiano $\mathbf{n} \sim \mathcal{N}(0, \sigma^2 \mathbf{I})$). Em vez de reconstruir o input ruidoso, a rede é treinada para recuperar o input original limpo, minimizando $\frac{1}{2}\|\hat{\mathbf{x}} - \mathbf{x}\|^2$. O sistema aprende a representar não apenas os pontos de dados, mas também as suas vizinhanças na variedade (manifold), tornando-se robusto a perturbações.

Stochastic Auto-encoders

Neste modelo, o mapeamento deixa de ser determinístico. Utilizam-se distribuições probabilísticas para o codificador $p_{\text{encoder}}(\mathbf{h}|\mathbf{x})$ e para o descodificador $p_{\text{decoder}}(\mathbf{x}|\mathbf{h})$. O objetivo foca-se na minimização da log-verosimilhança negativa $-\log p_{\text{decoder}}(\mathbf{x}|\mathbf{h})$.

9.8. Why Do We Use Auto-Encoders?

Historicamente, o treino de redes profundas era instável devido a problemas de convergência. Os Auto-encoders surgiram como uma solução para inicializar redes de forma inteligente, permitindo que estas captassem a estrutura dos dados antes de sequer verem os rótulos (labels).

Unsupervised Pre-training

Esta técnica segue um procedimento greedy layer-wise: Treina-se uma camada de cada vez, do início ao fim, utilizando um critério não supervisionado (como um Auto-encoder). Uma vez treinada uma camada, os seus parâmetros são fixados, e as suas saídas servem de input para a camada seguinte. Este processo coloca os parâmetros numa região do espaço de procura onde os ótimos locais resultam em menor overfitting quando o treino supervisionado começa.

Fine-Tuning

Após o pré-treino de todas as camadas, a rede é finalizada para a tarefa específica. Adiciona-se a camada final (ex: Softmax para classificação). Treina-se a rede completa utilizando o algoritmo de backpropagation convencional. Neste passo, todas as representações aprendidas anteriormente são "afinadas" para se tornarem mais discriminativas para a tarefa em questão.

Other applications

Para além do pré-treino, os Auto-encoders são ferramentas versáteis em diversas áreas:

- Redução de Dimensionalidade: Funcionam como uma alternativa não-linear à PCA, comprimindo dados mantendo a informação essencial.
- Recuperação de Informação e Semantic Hashing: Através da binarização dos códigos no bottleneck, é possível indexar documentos de forma a que itens semanticamente semelhantes tenham hashes próximos.
- Conversão de Inputs Discretos: Transformação de dados discretos (como palavras ou categorias) num espaço contínuo de baixa dimensão, facilitando o processamento por outros modelos.

9.9. Semântica Distribucional

A premissa fundamental para aprender estas representações é a Hipótese Distribucional: uma palavra é definida pelo contexto em que ocorre. Como afirmou J. R. Firth em 1957: "You shall know a word by the company it keeps". Palavras com significados semelhantes tendem a aparecer rodeadas pelas mesmas palavras (ex: "ler" aparece frequentemente perto de "livro" e "jornal"). Esta ideia permite transformar o significado semântico, que é abstrato, em relações estatísticas concretas.

Word Embeddings

O objetivo é obter vetores densos de baixa dimensão que capturem estas relações. Existem dois métodos principais. Fatorização de Matrizes: Decomposição de matrizes de co-ocorrência palavra-contexto (ex: LSA). Modelos Preditivos: Treinar uma rede neuronal para prever a palavra dado o contexto (ou vice-versa), como o word2vec.

9.10. Evolução: Do Modelo de Bengio ao word2vec

Neural Language Model (Bengio et al., 2003)

Este modelo pioneiro introduziu a Matriz de Embedding, onde cada palavra do vocabulário é associada a um vetor. Tarefa: Prever a próxima palavra dadas as K palavras anteriores.Processo: Os embeddings das palavras de contexto são concatenados e passados por uma rede neuronal.Saída: Uma camada Softmax gigante com o tamanho do vocabulário.Aprendizagem: O gradiente é propagado até à matriz de embedding, ajustando as coordenadas dos vetores.

word2vec (Mikolov et al. (2013))

Perceberam que, se o objetivo for apenas obter bons embeddings (e não um modelo de linguagem perfeito), o processo pode ser simplificado. Contexto não Direcional: Não precisamos de prever apenas da esquerda para a direita; podemos usar o contexto à esquerda e à direita simultaneamente. Eficiência: Não é necessário calcular a probabilidade de todas as palavras do vocabulário em cada passo (o que é computacionalmente caro), mas sim garantir que a palavra real seja mais provável do que uma palavra aleatória (Negative Sampling).

9.11. Word2Vec

O Word2Vec baseia-se em janelas de contexto e apresenta duas variantes principais. CBOW (Continuous Bag-of-Words): Prevê a palavra central com base nas palavras circundantes. Skip-Gram: Prevê as palavras do contexto (x_{t+j}) dada a palavra central (x_t). É a variante mais utilizada devido à sua eficácia em capturar relações semânticas mesmo com dados menos frequentes.

Skip-Gram

O objetivo é maximizar a probabilidade logarítmica de observar o contexto dado o centro:

$$J(\Theta) = \frac{1}{T} \sum_{t=1}^{T} \sum_{-m \leq j \leq m, j \neq 0} \log p(x_{t+j} | x_t)$$

Parametrização: Cada palavra possui dois vetores de embedding: $\mathbf{u}_o$ (quando é a palavra central) e $\mathbf{v}_c$ (quando atua como contexto). A probabilidade é modelada via log-bilinear (Softmax):

$$p(c|o) = \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{c'} \exp(\mathbf{u}_o^\top \mathbf{v}_{c'})}$$

Negative Sampling

Calcular o denominador da Softmax para cada passo de treino é proibitivo quando o vocabulário excede as 50.000 palavras. O Negative Sampling resolve isto transformando o problema de classificação multiclasse num problema de classificação binária. Ideia Central: Em vez de prever a probabilidade exata de uma palavra, o modelo aprende a distinguir entre um par real (centro, contexto) e k pares aleatórios (centro, ruído). A nova função de custo para um passo t torna-se:

$$J_t(\Theta) = \log \sigma(\mathbf{u}_o^\top \mathbf{v}_c) + \sum_{i=1}^{k} \log \sigma(-\mathbf{u}_o^\top \mathbf{v}_{j_i})$$

onde σ é a função sigmóide e j_i são palavras amostradas aleatoriamente (geralmente com base na frequência unigrama).

9.12. Outros Métodos: GloVe e FastText

nquanto o Word2Vec se foca em janelas locais, outros métodos exploram diferentes níveis de informação:

GloVe (Global Vectors): Treinado com base em estatísticas globais de co-ocorrência de palavras em todo o corpus. Em vez de prever contextos, o GloVe foca-se na razão entre probabilidades de co-ocorrência para capturar significado semântico.

FastText: Desenvolvido pela FAIR, estende o modelo de skip-gram ao decompor palavras em n-gramas de caracteres. Isto permite gerar embeddings para palavras fora do vocabulário (out-of-vocabulary) e lidar melhor com línguas morfologicamente ricas.

9.13. Problemas em Aberto e Contextualização

As embeddings tradicionais (Word2Vec, GloVe) são estáticas: cada palavra tem um único vetor, o que falha em capturar a polissemia (ex: "banco" como instituição vs. "banco" de jardim).

Embeddings Contextuais (ELMo). Surgido em 2018, o ELMo (Embeddings from Language Models) revolucionou o campo ao gerar representações dinâmicas. Mecanismo: Pré-treina um modelo de linguagem BiLSTM em larga escala. Funcionamento: Em vez de usar apenas uma tabela de consulta, a frase inteira é passada pelo codificador. As representações finais são uma combinação linear das camadas internas do modelo, ajustadas à tarefa específica (fine-tuning).

Embeddings Cross-Lingues. O desafio aqui é colocar palavras de diferentes línguas (ex: "cão" e "dog") no mesmo espaço vetorial. Espaço Partilhado: Pode ser alcançado através de dicionários bilingues e transformações lineares ou através de treino adversarial (Lample et al., 2018), que alinha os espaços vetoriais sem necessidade de tradução paralela. Sentenças Paralelas: Utilização de perdas que forçam representações de frases traduzidas a serem semanticamente próximas no espaço vetorial.

10. Lecture 7: Convolutional Neural Networks (CNNs)

10.1. CNN

Uma CNN é essencialmente uma rede neuronal com uma estrutura de conectividade especializada, inspirada na hierarquia do sistema visual biológico. A sua arquitetura é frequentemente descrita como uma arquitetura "sandwich", alternando entre:

- Células Simples (Camadas Convolucionais): Com parâmetros que podem ser aprendidos.
- Células Complexas (Camadas de Pooling): Que realizam operações de pooling para reduzir a dimensionalidade.

A principal distinção reside em como os neurónios se conectam aos inputs. Numa Camada Fully Connected, cada ativação depende de todos os inputs da camada anterior. Isto obriga a "esticar" ou achatar (flatten) a imagem (ex: transformar uma grelha 28x28 num vetor de 784 elementos), perdendo-se a informação sobre a vizinhança espacial.

Na camada convolucional preserva-se a estrutura espacial. Em vez de olhar para a imagem toda de uma vez, utilizamos filtros (ou kernels) que deslizam sobre o input. Localidade: Cada neurónio liga-se apenas a uma pequena região local do input. Partilha de Parâmetros: O mesmo filtro é aplicado em toda a imagem, o que reduz drasticamente o número de parâmetros e permite detetar a mesma característica (ex: uma aresta) em qualquer posição.

10.2. Hiperparâmetros da Convolução

O tamanho do output de uma camada convolucional é determinado por quatro variáveis principais: o tamanho do input (N), o tamanho do filtro (F), o stride (S) e o padding (P). Cálculo da Dimensão de Saída, dada uma imagem $N \times N \times D$ e K filtros de tamanho $F \times F \times D$, a dimensão espacial da saída $M \times M \times K$ é calculada como:

$$M = \frac{N - F + 2P}{S} + 1$$

Stride (S): O desdobramento (salto) do filtro sobre a imagem. Um stride maior reduz drasticamente a dimensão. Padding (P): Adição de zeros nas bordas para evitar a perda de informação periférica. O "Same Padding", onde $P = (F - 1)/2$, garante que $M = N$.

Matemática da Convolução e Partilha de Parâmetros

Embora na prática as bibliotecas de Deep Learning utilizem frequentemente a correlação cruzada, o termo "convolucional" advém do processamento de sinal. Para sinais finitos, a operação é:

$$h[t] = \sum_{a=-E}^{E} w[a]x[t - a]$$

Vantagens do Parameter Tying (Partilha de Pesos). Equivariância à Translação: Se uma característica (como um olho) se deslocar na imagem, o mapa de ativação desloca-se de igual forma. Eficiência Paramétrica: Reduz drasticamente o número de pesos a aprender comparado com camadas densas. Flexibilidade: Permite processar inputs de tamanhos variáveis.

Pooling

O segundo ingrediente da arquitetura "sandwich" é o Pooling. Esta camada opera de forma independente em cada canal (mapa de ativação). Max Pooling, consiste em selecionar o valor máximo dentro de uma janela (ex: 2×2). Invariância: Torna a rede robusta a pequenas variações, rotações ou escalas no input. Se um padrão se mover ligeiramente dentro da janela de pooling, a saída permanece a mesma. Redução de Dimensionalidade: Torna as representações menores e mais geríveis computacionalmente, focando-se apenas nas características mais fortes detetadas pelos filtros.

10.3. Evolução das Redes Convolucionais

LeNet-5 (LeCun et al., 1989)

Inspirada no "Neocognitron" de Fukushima, a LeNet foi a primeira implementação bem-sucedida de convoluções 2D. Introduziu a ideia de aplicar o mesmo filtro em todas as localizações da imagem com pesos partilhados. Estabeleceu o padrão de alternar camadas convolucionais com camadas de pooling para processar dígitos manuscritos (MNIST).

VGG-19 (Simonyan and Zisserman, 2014)

A VGG demonstrou que a profundidade era a chave para a performance.Utilizou filtros pequenos (3×3) de forma repetida.Embora muito profunda (19 camadas), o seu elevado número de parâmetros e operações (FLOPs) tornava-a pesada computacionalmente.

10.4. Redes Residuais (ResNets)

À medida que as redes se tornavam mais profundas, surgiu o problema do Desvanecimento do Gradiente (Vanishing Gradient), onde o sinal de erro se perdia antes de chegar às primeiras camadas.

Skip-Connections (Conexões de Atalho)

He et al. (2016) introduziram as skip-connections, que permitem que o gradiente flua diretamente através da rede sem passar apenas pelas transformações não-lineares.Seja $H(x) = F(x) + \lambda x$, o gradiente na retropropagação torna-se:

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial H} \left(\frac{\partial F}{\partial x} + \lambda \right)$$

Vantagens, Estabilidade: Mesmo que o gradiente da função aprendida $F(x)$ seja pequeno, o termo λ (geralmente 1) garante que o gradiente continue a ser propagado. Profundidade Extrema: Permitiu treinar redes com 152 camadas ou mais, superando significativamente a VGG em eficiência e precisão.

10.5. Convolutional Nets in NLP

Embora as CNNs sejam mais conhecidas pelo seu sucesso no processamento de imagens, estas desempenham um papel relevante no Processamento de Linguagem Natural (NLP). No contexto de NLP, as convoluções operam numa única dimensão (temporal/sequencial). O texto é tratado como uma sequência de vetores (word embeddings) onde os filtros deslizam ao longo do comprimento da frase. Para uma janela de contexto de tamanho $2h + 1$ centrada na palavra i, a ativação h_i é calculada concatenando os vetores das palavras vizinhas:

$$h_i = g(\mathbf{W}[\mathbf{x}_{i-h} \oplus \cdots \oplus \mathbf{x}_i \oplus \cdots \oplus \mathbf{x}_{i+h}] + \mathbf{b})$$

A matriz $\mathbf{W}$ é partilhada por todas as posições da frase, permitindo detetar a mesma característica semântica independentemente da posição em que ocorre.

10.6. Gestão de Sequências de Tamanho Variável

Ao contrário das imagens, que podem ser redimensionadas para um formato fixo, as frases têm comprimentos naturalmente distintos. Isto introduz desafios para o processamento em mini-batches nas GPUs.

Padding e Masking

Para agrupar frases de diferentes tamanhos num único tensor retangular, utilizamos Padding. Adicionam-se símbolos especiais (zeros) às frases mais curtas para que todas tenham o mesmo comprimento que a frase mais longa do batch. Masking, é essencial aplicar uma máscara para que o modelo "ignore" os valores de padding durante o cálculo da perda e do gradiente, garantindo que estes elementos artificiais não influenciam o resultado final.

Otimização de Batches

Uma estratégia comum para minimizar o desperdício computacional (excesso de zeros) é ordenar o dataset pelo comprimento das frases antes de formar os mini-batches. Assim, as frases dentro de um mesmo batch têm tamanhos semelhantes.

10.7. Além das Convoluções

Embora as CNNs dominem a visão computacional há anos, surgiram alternativas poderosas como os Transformers. Ao contrário das convoluções, que focam em janelas locais, os Transformers utilizam mecanismos de atenção para capturar relações globais entre qualquer parte do input (seja uma imagem ou uma frase). Este conceito, por vezes comparado a "convoluções dinâmicas", será o foco central de estudos sobre Modelos de Linguagem de Larga Escala (LLMs).

Visualização de Representações

Para compreender o que acontece dentro da "caixa negra", podemos utilizar técnicas de visualização. A ideia base é otimizar o input (a imagem) para maximizar a ativação de um neurónio ou camada específica, em vez de otimizar os pesos.

Hierarquia de Características

Maximizar camadas iniciais revela detetores de arestas e texturas; camadas mais profundas revelam formas complexas e objetos abstratos.

Ataques Adversários e Robustez

As redes neuronais, apesar da sua precisão, podem ser extremamente frágeis (brittle). Pequenas perturbações, muitas vezes impercetíveis ao olho humano, podem mudar drasticamente a classificação de um modelo. Ataques Glass-box: Assumem acesso total ao modelo. Utilizamos o backpropagation para encontrar os píxeis exatos que, se alterados, maximizam o erro ou forçam uma classe errada. Perturbação de Objetos Físicos: O perigo estende-se ao mundo real. Texturas alteradas em modelos 3D (como uma tartaruga impressa em 3D) podem enganar um classificador para que este veja uma arma, independentemente do ângulo de visão.

11. Lecture 8: Recurrent Neural Networks (RNNs)

11.1. Introdução às RNNs

Grande parte dos dados relevantes no mundo real tem natureza sequencial, como palavras em textos, sequências de DNA, retornos do mercado financeiro e sinais de áudio. O desafio central é como processar sequências que possuem comprimentos arbitrários.

Arquitetura: Feed-forward vs. Recurrente

A principal diferença entre estas arquiteturas reside na forma como gerem o estado interno e a dependência temporal:

- **Redes Feed-forward:** São modelos estáticos onde o estado oculto e a saída dependem apenas do input atual:

$$h = g(Vx + c) \tag{5}$$

$$\hat{y} = Wh + b \tag{6}$$

- **Redes Recorrentes (RNN):** Introduzidas por Elman (1990), funcionam como sistemas dinâmicos onde o estado atual (h_t) é influenciado pelo input presente (x_t) e pelo estado oculto anterior (h_{t-1}), comprimindo assim todo o histórico da sequência num vetor:

$$h_t = g(Vx_t + Uh_{t-1} + c) \tag{7}$$

$$\hat{y}_t = Wh_t + b \tag{8}$$

11.2. Treino e Retropropagação no Tempo (BPTT)

Para treinar os parâmetros de uma RNN, utiliza-se uma representação denominada "grafo desenrolado" (*unrolled graph*).

Backpropagation Through Time (BPTT)

O grafo desenrolado é um grafo de computação acíclico direcionado, o que permite a aplicação direta da retropropagação do gradiente.

- **Parameter Tying:** Os parâmetros da rede (W, U, V) são partilhados/amarrados através de todos os passos de tempo.

- **Agregação de Derivadas:** Uma vez que os parâmetros são partilhados, as derivadas são acumuladas ao longo de todos os passos temporais:

$$\frac{\partial \mathcal{L}}{\partial U} = \sum_{t=1}^{4} \frac{\partial h_t}{\partial U} \frac{\partial \mathcal{L}}{\partial h_t} \tag{9}$$

Este conceito de partilha de pesos é análogo ao funcionamento dos filtros em Redes Neuronais Convolucionais (CNNs).

11.3. Aplicações Padrão de RNNs

As Redes Neurais Recorrentes são aplicadas tipicamente em três cenários fundamentais:

1. **Geração de sequências:** gera símbolos sequencialmente com um modelo auto-regressivo (ex: modelagem de linguagem).

2. **Sequence tagging:** recebe uma sequência como entrada e retorna uma etiqueta para cada elemento da sequência (ex: *part-of-speech* (POS) tagging).

3. **Pooled classification:** recebe uma sequência como entrada e retorna um rótulo único através do *pooling* dos estados da RNN (ex: classificação de texto).

11.4. Modelos Auto-Regressivos

O objetivo destes modelos é considerar todo o histórico da sequência, comprimindo-o num vetor de estado, ao contrário dos modelos de Markov que utilizam memória limitada.

Ideias-Chave

- **Feedback de Saída:** A saída anterior é alimentada como entrada para o passo atual: $x_t =$ embedding de y_{t-1}.
- **Vetor de Estado:** Mantém-se um vetor de estado h_t, que é uma função do estado anterior e da entrada atual, comprimindo todo o histórico:

$$h_t = g(Vx_t + Uh_{t-1} + c) \tag{10}$$

- **Probabilidade de Saída:** Calcula-se a probabilidade do próximo símbolo:

$$P(y_t = i \mid y_{t-1}, \ldots, y_0) = [\text{softmax}(Wh_t + b)]_i \tag{11}$$

Modelagem de Linguagem

Para gerar texto, cada y_t corresponde a uma palavra num vocabulário tipicamente vasto ($|V| \approx 10^5$). Definindo $z_t = Wh_t + b$, a probabilidade é dada por:

$$p(y_t = i \mid y_{t-1}, \ldots, y_0) = \frac{\exp((z_t)_i)}{\sum_j \exp((z_t)_j)} = [\text{softmax}(z)]_i \tag{12}$$

A probabilidade conjunta da sequência é calculada através da auto-regressão:

$$\begin{aligned} P(y_1, \ldots, y_L) &= P(y_1) \times P(y_2 \mid y_1) \times \cdots \times P(y_L \mid y_{L-1}, \ldots, y_1) \\ &= (\text{softmax}(Wh_1 + b))_{y_1} \times (\text{softmax}(Wh_2 + b))_{y_2} \times \ldots \\ &\quad \times (\text{softmax}(Wh_L + b))_{y_L} \end{aligned} \tag{13}$$

11.5. Três Problemas para RNNs de Geração de Sequências

Para a implementação de modelos recorrentes generativos, são necessários algoritmos que resolvam três tarefas fundamentais:

- **Amostragem de sequências** a partir da distribuição de probabilidade definida pela RNN.
- **Obtenção da sequência mais provável**.
- **Treino da RNN**, o que envolve a aprendizagem dos parâmetros W, U, V, b e c.

Amostragem de uma Sequência

Este processo é computacionalmente simples. Utilizando a notação onde y_t representa o *embedding* da palavra y_t, o algoritmo segue os seguintes passos:

- Calcular h_1 a partir de $x_1 = \text{START}$.
- Amostrar $y_1 \sim \text{softmax}(Wh_1 + b)$.
- Calcular h_2 a partir de h_1 e definir a próxima entrada como $x_2 = y_1$.
- Amostrar $y_2 \sim \text{softmax}(Wh_2 + b)$.
- Repetir o processo até atingir um critério de paragem.

Obtenção da Sequência Mais Provável

Encontrar a sequência globalmente mais provável é um problema complexo:

- O objetivo é encontrar a sequência $y_1, y_2, \ldots$ que maximize conjuntamente o produto das probabilidades:

$$(\text{softmax}(Wh_1 + b))_{y_1} \times (\text{softmax}(Wh_2 + b))_{y_2} \times \ldots$$

- Uma abordagem gananciosa (*greedy*), que escolhe o melhor y_t em cada passo de forma isolada, não funciona, pois a distribuição em cada instante depende de todas as escolhas anteriores.
- Este problema é particularmente relevante em modelos de linguagem condicional, como a tradução automática.

Treino da RNN

O treino de RNNs para geração de sequências baseia-se tipicamente na **Estimação de Máxima Verosimilhança**. O objetivo é minimizar a função de perda *log-loss* (entropia cruzada):

$$L(\Theta; y_1, \ldots, y_L) = -\frac{1}{L+1} \sum_{t=1}^{L+1} \log P_\Theta(y_t \mid y_0, \ldots, y_{t-1})$$

Nesta formulação, Θ representa o conjunto de parâmetros (W, U, V, b, c).

- **Perplexidade:** Minimizar esta função de perda é equivalente a minimizar a perplexidade do modelo, definida por $\exp(L(\Theta, y_{1:L}))$.
- **Intuição:** O valor de $-\log P_\Theta$ quantifica o nível de "surpresa" ou perplexidade do modelo ao encontrar a palavra real na sequência.
- **Capacidade de Memória:** Diferente dos modelos de Markov de ordem fixa, uma RNN tem o potencial de reter informação de todo o histórico, embora a aprendizagem eficaz dessa memória seja um desafio prático.

11.6. Teacher Forcing e Bias de Exposição

No treino de modelos recorrentes, utiliza-se frequentemente uma técnica de condicionamento específica:

- **Teacher Forcing:** O modelo é condicionado no histórico real (alvo) em cada passo de tempo, em vez de utilizar as suas próprias previsões anteriores.
- **Exposure Bias:** O uso de *teacher forcing* causa um viés de exposição. Em tempo de execução (*runtime*), o modelo pode ter dificuldade em recuperar de erros, pois gera histórias e contextos que nunca encontrou durante a fase de treino.

11.7. Modelagem de Linguagem ao Nível do Caractere

É possível implementar RNNs que operam sobre caracteres em vez de palavras inteiras.

Vantagens

Permite gerar qualquer combinação de caracteres (resolvendo o problema de palavras fora do vocabulário) e possui um conjunto de símbolos de saída muito menor, resultando numa camada *softmax* reduzida.

Desvantagem

O modelo necessita de reter informação de um histórico muito mais profundo para capturar dependências semânticas.

11.8. Etiquetagem de Sequências (Sequence Tagging)

Na etiquetagem de sequências, o objetivo é atribuir uma etiqueta a cada elemento de uma sequência de entrada $x_1, \ldots, x_L$, gerando uma sequência de saída $y_1, \ldots, y_L$. Exemplos comuns incluem a etiquetagem gramatical (POS tagging) e o reconhecimento de entidades nomeadas.

Diferenças face à Geração de Sequências

- O input e o output são distintos, eliminando a necessidade de auto-regressão.
- O comprimento da sequência de saída é conhecido, sendo idêntico ao da entrada.

Tagger POS Baseado em RNN

O modelo processa a sequência da seguinte forma:

- **Entradas:** Os inputs são *word embeddings* obtidos a partir de uma matriz de *embedding* de dimensão $V \times E$.
- **Vetor de Estado:** Mantém-se um estado oculto que comprime o histórico do input até ao instante t:
$$h_t = g(Vx_t + Uh_{t-1} + c)$$
- **Camada de Saída:** Uma camada *softmax* computa a probabilidade da etiqueta atual dado o contexto:
$$P(y_t \mid x_1, \ldots, x_t) = (\text{softmax}(Wh_t + b))_{y_t}$$

11.9. RNNs Bidirecionais e Reconhecimento de Entidades

As sequências podem ser processadas em diferentes sentidos para enriquecer a representação:

- **Processamento:** Uma sequência pode ser lida da esquerda para a direita, da direita para a esquerda, ou em ambas as direções combinando as representações.
- **NER (Named Entity Recognition):** O modelo de RNN utilizado para *POS tagging* é aplicável ao NER. Contudo, o NER possui restrições de transição de etiquetas (ex: não se pode ter I-PER após B-LOC).
- **Limitação:** O modelo de etiquetagem por RNN explora a estrutura de entrada através dos estados, mas carece de estrutura na saída para lidar com estas restrições.

11.10. Classificação Agrupada (Pooled Classification)

Para além de sequências de etiquetas, as RNNs podem ser usadas para prever um rótulo único para toda a sequência.

Estratégia Geral

O modelo captura a estrutura sequencial da entrada e realiza o *pooling* (agrupamento) dos estados da RNN, mapeando-os para um vetor único que alimenta uma *softmax* final.

Estratégias de Pooling

- **Último estado:** Utilizar o último estado da RNN, que contém informação de toda a travessia da sequência. Apresenta a desvantagem de que, em sequências longas, a influência das primeiras palavras pode desaparecer.
- **Alternativas:** Utilizar uma RNN bidirecional combinando os estados finais de ambos os sentidos, realizar *average pooling* ou outras técnicas de agregação.

11.11. Retropropagação no Tempo e Gradientes

O comportamento dos gradientes ao retroceder no tempo é crítico para o treino de RNNs.

Cálculo do Gradiente

A derivada da função em relação ao primeiro estado oculto envolve o produto das derivadas parciais entre estados:

$$\frac{\partial \mathcal{F}}{\partial h_1} = \frac{\partial h_2}{\partial h_1}\frac{\partial h_3}{\partial h_2}\frac{\partial h_4}{\partial h_3}\frac{\partial \hat{y}}{\partial h_4}\frac{\partial \mathcal{F}}{\partial \hat{y}}$$

Sendo que $\prod_t \frac{\partial h_t}{\partial h_{t-1}} = \prod_t \mathrm{Diag}(g'(z_t))U$.

Cenários de Estabilidade

Dependendo do maior valor próprio da matriz de pesos U:

- **Exatamente 1:** A propagação do gradiente é estável.
- **Menor que 1:** O gradiente desaparece (*vanishing*), ocorrendo um decaimento exponencial.
- **Maior que 1:** O gradiente explode (*exploding*), ocorrendo um crescimento exponencial.

11.12. Soluções para Vanishing e Exploding Gradients

Gradient Clipping

Técnica usada para lidar com a explosão de gradientes através da sua truncagem.

- **Norm clipping:** $\nabla \leftarrow \frac{c}{\|\nabla\|}\nabla$ se $\|\nabla\| \geq c$.
- **Elementwise clipping:** $\tilde{\nabla}_i \leftarrow \min\{c, |\nabla_i|\} \times \mathrm{sign}(\nabla_i)$.

Mitigação do Desvanecimento

O desvanecimento é mais frequente e torna difícil a aprendizagem de dependências de longo prazo. As soluções incluem:

- Melhores otimizadores (métodos de segunda ordem).
- Normalização para manter as normas dos gradientes estáveis.
- Inicialização cuidada (ex: matrizes ortogonais aleatórias).
- Parametrizacões alternativas como **LSTMs** e **GRUs**.

11.13. RNNs Alternativas: GRU e LSTM

As arquiteturas de portas surgiram com a intuição de que, em vez de multiplicar o erro através do tempo (o que leva ao crescimento ou decaimento exponencial), o erro deve manter-se aproximadamente constante. Estas redes resolvem o problema do desvanecimento do gradiente, embora ainda possam sofrer de explosão de gradiente, exigindo o uso de *gradient clipping*.

11.14. Gated Recurrent Units (GRU)

A ideia central da GRU é criar conexões de atalho (*shortcut connections*) adaptativas controladas por portas especiais para facilitar a retropropagação através dos nós intermédios.

Formulação Matemática

A dinâmica da GRU é governada pelas seguintes equações:

- **Update gate (u_t):** $\sigma(V_u x_t + U_u h_{t-1} + b_u)$
- **Reset gate (r_t):** $\sigma(V_r x_t + U_r h_{t-1} + b_r)$
- **Candidato a atualização ($\tilde{h}_t$):** $g(V x_t + U(r_t \odot h_{t-1}) + b)$
- **Estado oculto final (h_t):** $u_t \odot \tilde{h}_t + (1 - u_t) \odot h_{t-1}$

11.15. Long Short-Term Memories (LSTM)

A característica fundamental da LSTM é a utilização de células de memória (c_t). Para evitar o efeito multiplicativo, a informação flui de forma aditiva através destas células.

Mecanismo de Portas

O fluxo de informação é controlado por três portas de ativação sigmoide:

- **Forget gate (f_t):** $\sigma(V_f x_t + U_f h_{t-1} + b_f)$
- **Input gate (i_t):** $\sigma(V_i x_t + U_i h_{t-1} + b_i)$
- **Output gate (o_t):** $\sigma(V_o x_t + U_o h_{t-1} + b_o)$

Atualização de Estado

A transição da memória e do estado oculto é definida por:

- **Célula de memória (c_t):** $f_t \odot c_{t-1} + i_t \odot g(Vx_t + Uh_{t-1} + b)$
- **Estado oculto (h_t):** $o_t \odot g(c_t)$

11.16. LSTMs Bidirecionais

As LSTMs Bidirecionais aplicam a mesma lógica das RNNs bidirecionais, mas utilizam unidades LSTM em vez de células recorrentes padrão para capturar contextos passados e futuros de forma mais robusta.

11.17. Das Sequências às Árvores

Para além da estrutura sequencial 1D, é possível lidar com outras topologias, como árvores, utilizando computação recursiva através de RNNs.

Redes Neurais Recursivas

Estas redes operam propagando estados de baixo para cima (*bottom-up*) numa árvore, tipicamente binária, onde cada nó, exceto as folhas, tem dois filhos. O estado do nó pai p é computado a partir dos estados dos filhos c_1 e c_2:

$$p = \tanh\left(W \begin{bmatrix} c_1 \\ c_2 \end{bmatrix} + b\right)$$

- **Partilha de Parâmetros:** Utilizam-se os mesmos parâmetros W e b em todos os nós da árvore.
- **Saídas:** Podem ser calculadas pontuações na raiz ou em cada nó através da adição de uma camada de saída *softmax* nesses nós.
- **Tree-LSTMs:** Estendem as redes recursivas utilizando portas (*gates*) para lidar com os filhos esquerdo e direito, de forma análoga à extensão das RNNs para LSTMs.

11.18. Extensões para Imagens e Outras Dimensões

Embora as sequências sejam unidimensionais, as imagens possuem uma estrutura bidimensional.

- **PixelRNNs:** São extensões 2D das RNNs que funcionam como modelos auto-regressivos para gerar imagens píxel a píxel, condicionando cada um aos píxeis vizinhos numa ordem específica.

11.19. Truques do Ofício (*Tricks of the Trade*)

Para obter alto desempenho e velocidade no treino de modelos recorrentes profundos, aplicam-se várias técnicas práticas.

Profundidade e Dropout

- **Camadas Profundas:** O empilhamento de 2 a 8 camadas recorrentes é uma prática padrão que auxilia no desempenho do modelo.
- **Dropout:** Deve ser aplicado entre as camadas, mas não nas conexões recorrentes. Em alternativa, pode-se utilizar a mesma máscara de *dropout* para todas as conexões recorrentes.

Otimização e Velocidade

- **Implementação de Portas:** O uso de matrizes diagonais em vez de matrizes completas pode acelerar o cálculo, especialmente nas portas.
- **Concatenação:** Concatenar as matrizes de parâmetros de todas as portas permite realizar uma única multiplicação matriz-vetor em vez de várias, aumentando a eficiência.
- **Inicialização:** Recomenda-se inicializar o *bias* da porta de esquecimento (*forget gate*) com um valor elevado para garantir o fluxo de sinal no início do treino. Devem ser usadas matrizes ortogonais aleatórias para inicializar as matrizes quadradas.

Mini-Batching em Sequências

- **Eficiência:** O *mini-batching* permite converter múltiplos produtos matriz-vetor numa única operação matriz-matriz, otimizando o uso do hardware.
- **Gestão de Comprimento:** Como as sequências têm tamanhos variáveis, a solução consiste em agrupar instâncias de comprimentos semelhantes (*bucketing*) e aplicar preenchimento com zeros (*padding*).
- **Cuidado com o Padding:** É necessário assegurar que os valores de *padding* não resultem na retropropagação de valores não nulos que afetem o treino.

12. Lecture 9: Sequence-to-Sequence Models

12.1. Sequence-to-Sequence (Seq2Seq)

Os modelos *sequence-to-sequence* mapeiam uma sequência de origem para uma sequência de destino, ambas de comprimentos arbitrários. Esta abordagem difere da etiquetagem de sequências (*sequence tagging*), onde as sequências de entrada e saída têm obrigatoriamente o mesmo comprimento.

12.2. Tradução Automática Estatística (SMT)

Entre as décadas de 1990 e 2010, o objetivo da SMT era encontrar a frase de destino y mais provável dada uma frase de origem x:

$$\hat{y} = \arg\max_{y} P(y \mid x)$$

Utilizando a Lei de Bayes, este problema é decomposto em dois componentes principais:

$$\hat{y} = \arg\max_{y} P(x \mid y)P(y)$$

- **Modelo de Tradução ($P(x \mid y)$):** Modela como as palavras ou frases são traduzidas, sendo aprendido a partir de dados paralelos.
- **Modelo de Linguagem ($P(y)$):** Modela a fluidez da língua de destino, sendo aprendido a partir de grandes volumes de dados monolingues.

12.3. Alinhamento de Palavras e Modelos IBM

Para aprender o modelo de tradução $P(x \mid y)$, os Modelos IBM introduziram a variável latente de alinhamento a, que representa as correspondências ao nível da palavra entre as duas frases.

Tipos de Alinhamento

Os alinhamentos podem assumir diversas formas:

- **Um-para-muitos (Fertilidade):** Uma palavra na origem gera várias palavras no destino.
- **Muitos-para-um:** Várias palavras na origem correspondem a apenas uma no destino.
- **Muitos-para-muitos:** Correspondências ao nível da frase (*phrase-based MT*).

Formulações dos Modelos IBM

O modelo de tradução é calculado marginalizando sobre todos os alinhamentos possíveis:

$$P(x \mid y) = \sum_a P(x, a \mid y)$$

Os modelos IBM (1, 2, 3, 4, ...) aprendem $P(x, a \mid y)$ combinando fatores como a probabilidade de co-ocorrência de palavras, a posição relativa e a fertilidade das palavras.

12.4. Procura e Descodificação

A procura da melhor tradução envolve resolver:

$$\hat{y} = \arg\max_y \sum_a P(x, a \mid y) P(y)$$

Uma vez que a enumeração de todas as hipóteses e alinhamentos é intratável, utiliza-se tipicamente uma **procura heurística** para construir a tradução gradualmente, descartando hipóteses com baixa probabilidade ao longo do processo.

12.5. Resumo: Tradução Automática Estatística (SMT)

A SMT clássica caracterizava-se por sistemas de extrema complexidade, baseados em *pipelines* com múltiplos subcomponentes desenhados separadamente. As principais limitações incluíam:

- Engenharia de características (*feature engineering*) exaustiva.
- Dependência excessiva do par de línguas no design do sistema.
- Elevado consumo de memória e armazenamento (ex: tabelas de frases).
- Grande esforço humano de manutenção e compilação de recursos.

12.6. Tradução Automática Neuronal (NMT)

A NMT representa uma mudança de paradigma ao utilizar uma única rede neuronal para realizar a tradução de ponta a ponta (*end-to-end training*), eliminando os *pipelines* complexos.

Arquitetura Encoder-Decoder

O modelo *sequence-to-sequence* baseia-se em dois componentes principais:

1. **Encoder RNN:** Codifica a frase de origem x e gera um vetor de estado (vetor de contexto c) que resume o seu significado.
2. **Decoder RNN:** Atua como um modelo de linguagem condicionado pelo vetor c para gerar a frase de destino y: $y \mid x \sim \text{RNNLM}(c)$.

12.7. Beam Search

Uma vez que enumerar todas as sequências possíveis y para encontrar a de probabilidade máxima é intratável, utiliza-se a **Beam Search** como estratégia de procura aproximada:

- Em cada passo do descodificador, mantêm-se as k traduções parciais mais prováveis (onde k é o *beam size*).
- Se $k = 1$, o processo equivale a uma *greedy search*.
- Oferece um compromisso (*trade-off*) linear entre velocidade e precisão. Na prática, utilizam-se tamanhos entre 4 e 12.

Truques Adicionais

- **LSTMs Profundas:** Aumentam a capacidade de representação.
- **Inversão da Frase de Origem:** Ajuda a estabelecer conexões mais próximas entre o início da frase de origem e o início da frase de destino.
- **Ensembling:** Combinar as previsões de N modelos treinados independentemente para obter um consenso.

12.8. Estado da Arte e Limitações

A NMT tornou-se o padrão da indústria em 2016 (ex: Google Translate), superando sistemas SMT desenvolvidos ao longo de décadas com apenas alguns meses de treino. No entanto, o problema não está resolvido:

Desafios em Aberto

- Palavras fora do vocabulário (OOV).
- Desfasamento de domínio entre treino e teste.
- Pares de línguas com poucos recursos (*low-resource*).
- Domínios específicos e técnicos (médico, legal).

Limitação Conceptual

Existe um limite físico na capacidade de um vetor fixo representar frases de comprimento arbitrário. Como referido por Ray Mooney: "Não se pode espremer o significado de uma frase inteira num único vetor". Este problema de "gargalo" motivou o desenvolvimento de mecanismos de atenção.

12.9. Codificação de Sentenças como Matrizes

Para resolver o problema do "gargalo" (*bottleneck*) e a incapacidade de um vetor fixo representar frases de comprimentos variados, a solução consiste em codificar a frase de origem como uma matriz.

Estratégias de Codificação

A matriz resultante possui um número fixo de linhas, mas o número de colunas depende da quantidade de palavras na frase.

- **Embeddings Simples:** Usar apenas vetores de palavras como colunas não é eficaz, pois carecem de contexto.
- **CNNs e LSTMs Bidirecionais:** Capturam o contexto local e global. O padrão atual (Bahdanau et al., 2015) utiliza **LSTMs Bidirecionais** para gerar esta matriz de representação F.

12.10. Mecanismo de Atenção

A atenção é a contraparte neuronal dos alinhamentos de palavras. Permite ao descodificador focar-se em partes específicas da matriz de entrada F ao gerar cada palavra de destino.

Funcionamento Matemático

Sejam $s_1, s_2, \ldots$ os estados do descodificador. Ao prever a t-ésima palavra:

1. **Similaridade (Energia):** Calcula-se a "proximidade" entre o estado anterior s_{t-1} e cada palavra da origem h_i:

$$z_{t,i} = v^\top g(Wh_i + Us_{t-1} + b)$$

2. **Distribuição de Atenção:** Normalização via *softmax* para obter pesos a_t:

$$a_t = \text{softmax}(z_t)$$

3. **Vetor de Contexto (c_t):** Soma pesada das colunas da matriz F (pooling pesado):

$$c_t = Fa_t = \sum_i a_{t,i} h_i$$

4. **Atualização de Estado:** O novo estado s_t é função do estado anterior, do símbolo anterior e do vetor de contexto:

$$s_t = \text{RNN}(s_{t-1}, [E(y_{t-1}), c_t])$$

12.11. Vantagens da Atenção

A introdução da atenção revolucionou a tradução automática neuronal por vários motivos:

- **Fim do Gargalo:** O descodificador olha diretamente para toda a frase de origem, não dependendo apenas de um vetor fixo.
- **Gradientes Estáveis:** Fornece "atalhos" para estados distantes, mitigando o problema do desvanecimento do gradiente.
- **Interpretabilidade:** Permite visualizar em que palavras o modelo se focou para gerar cada termo da saída.
- **Alinhamento Implícito:** A rede aprende a alinhar palavras sem nunca ter sido explicitamente treinada para tal (*end-to-end*).

Atenção vs. Alinhamento Clássico

Diferente dos modelos IBM, o modelo de Bahdanau original é uma versão "suave" (*soft*) de alinhamento e não possui enviesamentos estruturais (como saltos curtos ou fertilidade), embora extensões posteriores tentem adicionar estas restrições.

12.12. Outras Variantes e Extensões de Atenção

Para além do mecanismo de atenção padrão (Bahdanau), surgiram extensões que procuram aproximar o modelo dos alinhamentos clássicos e resolver problemas específicos da tradução automática neuronal, como as repetições excessivas de palavras.

Atenção Interpretável e Alinhamentos Rígidos

O mecanismo de *softmax* tradicional atribui sempre uma probabilidade não nula a todas as palavras da frase de origem, o que pode tornar a interpretação ruidosa. Procuram-se alternativas que permitam focar-se apenas num subconjunto relevante de palavras:

- **Atenção Esparsa (Sparsemax):** Em vez de utilizar a função *softmax*, utiliza-se o *sparsemax*. Esta função é capaz de atribuir exatamente zero de probabilidade a palavras irrelevantes, resultando numa distribuição de atenção esparsa e mais próxima dos "alinhamentos rígidos" (*hard alignments*).

Atenção com Restrições (Constrained Attention)

Um problema comum em NMT é o modelo focar-se repetidamente na mesma palavra da origem, gerando traduções com palavras duplicadas ou ignorando partes da frase.

- **Limites Superiores (Upper Bounds):** É possível impor um limite máximo à quantidade total de atenção que cada palavra da origem pode receber ao longo de toda a descodificação.
- **Prevenção de Repetições:** Ao limitar a "energia" acumulada de cada palavra (baseado na sua fertilidade), o modelo é forçado a mover o seu foco para outras partes da frase de origem assim que uma palavra tenha sido suficientemente "traduzida".

- **Softmax/Sparsemax Restrito:** Estas funções de ativação modificadas garantem que a soma da atenção recebida por cada palavra i ao longo do tempo não exceda um orçamento pré-definido.

13. Lecture 10: Attention Mechanisms and Transformers

13.1. Limitações das RNNs e a Transição para a Atenção

Apesar da eficácia das LSTMs e GRUs, as RNNs apresentam obstáculos fundamentais que motivaram a procura de novas arquiteturas:

- **Processamento Sequencial:** O mecanismo passo-a-passo proíbe a paralelização, tornando o treino lento em hardware moderno (GPUs).
- **Dependências de Longo Prazo:** Mesmo com o uso de portas, capturar relações entre palavras muito distantes continua a ser um desafio computacional.

A Pergunta Fundamental

Se o mecanismo de atenção permite o acesso direto a qualquer estado da sequência, eliminando a necessidade de percorrer todos os estados intermédios, será que precisamos realmente da componente recorrente (RNN)? Esta questão serve de base para o desenvolvimento do *Transformer*.

13.2. Mecanismo de Atenção: Recapitulando e Generalizando

A atenção permite que a rede atribua pesos de relevância aos inputs de forma automática, com a vantagem de ser rápida de paralelizar e fornecer uma ferramenta de interpretabilidade.

Cálculo de Afinidade (Scores)

Existem várias formas de "comparar" um vetor de consulta (*query*) q e um vetor de entrada h_i:

- **Atenção Aditiva:** $s_i = u^\top \tanh(Ah_i + Bq)$
- **Atenção Bilinear:** $s_i = q^\top U h_i$
- **Produto Escalar (Dot-product):** $s_i = q^\top h_i$ (mais eficiente para processamento em bloco/*batching*).

Chaves, Valores e Consultas (Key, Value, Query)

Na versão mais geral da atenção, os vetores de entrada assumem papéis distintos:

- **Query (q):** O vetor que representa o que procuramos (ex: estado do descodificador).
- **Keys (K):** Vetores usados para comparar com a *query* e obter os *scores* de afinidade.
- **Values (V):** Vetores que formam a média pesada final.

$$c = \sum_{i=1}^{L} p_i v_i, \quad \text{onde } p = \text{softmax}(s)$$

13.3. Self-Attention (Auto-Atenção)

Enquanto a atenção contextual foca-se na relação entre o descodificador e o codificador, a **self-attention** permite que cada posição da sequência atenda a todas as outras posições da própria sequência (seja no codificador ou no descodificador).

Camada de Self-Attention

Para uma sequência de comprimento L, o cálculo é realizado de forma matricial, o que permite uma paralelização total:

1. Projetar a entrada em matrizes de consulta (Q), chaves (K) e valores (V).
2. Calcular a matriz de afinidade (produto escalar): $S = QK^\top \in \mathbb{R}^{L \times L}$.
3. Converter os *scores* em probabilidades (por linha): $P = \text{softmax}(S)$.
4. Calcular a saída final:
$$Z = PV = \text{softmax}(QK^\top)V$$

Esta formulação permite capturar o contexto global da frase num único passo de computação, sem a necessidade de recorrência temporal.

13.4. A Arquitetura Transformer

A ideia central do Transformer é substituir a recorrência (RNN) e a convolução (CNN) por mecanismos de **self-attention** no codificador e no descodificador.

- No codificador, cada estado de palavra atende a todas as outras palavras da frase.
- Cada bloco de auto-atenção é seguido por uma transformação *feed-forward*.
- Este processo é repetido ao longo de várias camadas (tipicamente 6 ou 12).
- No descodificador, o processo é semelhante, mas a atenção é restrita às palavras já geradas (*masked attention*).

13.5. Scaled Dot-Product Attention

Ao utilizar o produto escalar para calcular a afinidade, surge um problema: à medida que a dimensão d_K aumenta, a variância do produto $QK^\top$ cresce, tornando a função *softmax* muito "picuda" e resultando em gradientes extremamente pequenos.

Solução: Escalonamento

A solução consiste em escalar o produto escalar pelo inverso da raiz quadrada da dimensão das chaves:
$$Z = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_K}}\right)V$$

13.6. Multi-Head Attention

Enquanto a auto-atenção simples funciona como uma convolução 1D com pesos adaptativos, ela oferece apenas um "canal" de interação entre as palavras. A **Multi-Head Attention** resolve esta limitação ao permitir múltiplos canais de atenção paralelos.

- Definem-se h cabeças de atenção (ex: $h = 8$), cada uma com as suas próprias matrizes de projeção W_i^Q, W_i^K, W_i^V.
- As saídas de cada cabeça (Z_i) são concatenadas e passadas por uma camada linear final:

$$\text{MultiHead}(X) = \text{Concat}(Z_1, \ldots, Z_h)W^O$$

Onde cada $Z_i = \text{Attention}(XW_i^Q, XW_i^K, XW_i^V)$.

13.7. Positional Encodings

Uma característica fundamental do Transformer é que ele é intrinsecamente insensível à ordem das palavras, uma vez que as consultas atendem às chaves independentemente da sua posição.

Estratégias de Codificação Posicional

Para introduzir a noção de ordem, adicionam-se **positional encodings** aos *embeddings* de entrada:

- **Aprendidas:** Treinar um *embedding* específico para cada posição.
- **Sinusoidais:** Utilizar funções seno e cosseno de diferentes frequências para codificar a posição absoluta e relativa:

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}})$$
$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

13.8. Outros Componentes Críticos

Para estabilizar o treino de redes tão profundas, o Transformer utiliza:

- **Conexões Residuais:** Aplicadas em cada bloco de atenção e *feed-forward*.
- **Layer Normalization:** Normalização aplicada após as conexões residuais para manter a escala das ativações.

13.9. O Descodificador (Decoder)

O descodificador do Transformer partilha a maioria dos componentes do codificador, mas introduz duas modificações cruciais para o funcionamento da geração de sequências:

- **Máscara Causal (Causal Masking):** Ao contrário do codificador, o descodificador não pode ter acesso a palavras futuras na sequência de destino. Para garantir que a previsão da palavra no instante t dependa apenas das palavras geradas nos instantes $1, \ldots, t-1$, aplica-se uma máscara na matriz de afinidade de auto-atenção, definindo os pesos das posições futuras como $-\infty$ antes da *softmax*.
- **Atenção Contextual (Cross-Attention):** Além da auto-atenção, o descodificador inclui um bloco adicional de atenção que consulta os estados finais produzidos pelo codificador. Neste caso, as *queries* provêm da camada anterior do descodificador, enquanto as *keys* e *values* provêm da saída do codificador.

13.10. Otimização e Truques Adicionais

Os modelos Transformer são conhecidos por serem mais sensíveis e difíceis de otimizar do que as RNNs tradicionais, exigindo um ajuste de hiperparâmetros rigoroso.

Técnicas de Regularização e Treino

- **Label Smoothing:** Substitui-se a codificação *one-hot* das etiquetas por uma versão suavizada que distribui uma pequena fração da probabilidade pelas palavras incorretas, prevenindo o excesso de confiança do modelo.
- **Dropout Estratégico:** Aplicado em todas as camadas imediatamente antes das conexões residuais.
- **Otimizador Adam com Decaimento:** Utiliza-se o otimizador Adam com um esquema de *warm-up* e posterior decaimento da taxa de aprendizagem (*learning rate*).

Estratégias de Descodificação

- **Beam Search com Penalização de Comprimento:** Como a *beam search* padrão tende a favorecer frases curtas, aplica-se uma penalização para normalizar a pontuação com base no comprimento da sequência gerada.

13.11. Resumo Comparativo: RNNs vs Transformers

- **Paralelização:** As RNNs são sequenciais ($O(L)$), enquanto o Transformer permite paralelização total ($O(1)$) no tempo de treino.
- **Custo por Camada:** A auto-atenção tem uma complexidade de $O(L^2 \cdot d)$, o que a torna eficiente para sequências curtas mas pesada para sequências muito longas face às RNNs ($O(L \cdot d^2)$).
- **Dependências:** O Transformer captura dependências de longo prazo em tempo constante ($O(1)$) devido à conectividade total, enquanto as RNNs requerem $O(L)$ passos.

14. Lecture 11: Self-Supervised Learning and Large Pretrained Models

14.1. Tokenização e Representações de Subpalavras

A utilização de vocabulários estáticos ao nível da palavra apresenta limitações, nomeadamente a dificuldade em lidar com vocabulários arbitrariamente grandes, palavras desconhecidas (*out-of-vocabulary*) e palavras compostas. A solução consiste em decompor as palavras em unidades menores: as **subpalavras**.

Byte Pair Encoding (BPE)

O BPE é um algoritmo de segmentação de subpalavras que atua como um híbrido entre representações ao nível do caractere e da palavra.

- **Ideia-chave:** Pares de subpalavras que ocorrem frequentemente são fundidos numa única unidade.

- **Algoritmo:**
  1. Inicializar o vocabulário com caracteres individuais.
  2. Contar a frequência de pares consecutivos de caracteres/subpalavras no *corpus*.
  3. Fundir o par mais frequente num novo símbolo.
  4. Repetir até atingir o tamanho de vocabulário pretendido ou um número máximo de fusões.

Alternativas e Limitações

Outros métodos de tokenização incluem o **Wordpiece** (usado no BERT), que funde pares com base no aumento da verosimilhança, e o **Unigram LM**, que remove subpalavras de forma iterativa. **Nota sobre LLMs:** Como modelos como o GPT utilizam BPE, eles não possuem uma consciência explícita de caracteres individuais, o que explica dificuldades em tarefas como inverter *strings* ou contar caracteres.

14.2. De Embeddings Estáticos a Contextualizados

Anteriormente, explorámos representações como Word2Vec ou GloVe, onde cada palavra possui um vetor único fixo, independentemente do contexto (**embeddings estáticos**). Contudo, a mesma palavra pode ter significados distintos conforme a frase.

Contextualização

As representações modernas são dinâmicas. O vetor de uma palavra é gerado em função de toda a sequência em que está inserida, permitindo capturar polissemia e nuances sintáticas.

14.3. Embeddings from Language Models (ELMo)

O ELMo foi um marco na transição para representações contextuais, utilizando uma arquitetura baseada em BiLSTMs.

Metodologia

1. **Pré-treino:** Treina-se um modelo de linguagem BiLSTM num conjunto de dados massivo.
2. **Extração de Parâmetros:** Ao contrário dos modelos estáticos, o ELMo guarda os parâmetros de todas as camadas, não apenas os *embeddings* iniciais.
3. **Adaptação:** Para uma tarefa específica, ajusta-se um parâmetro escalar para cada camada. A representação final de uma palavra é uma combinação linear das ativações de todas as camadas internas da rede.

14.4. Pré-treino e Aprendizagem Auto-Supervisionada

A aprendizagem auto-supervisionada (*self-supervised learning*) permite aproveitar vastas quantidades de texto sem anotações humanas. O modelo aprende a recuperar informação que foi deliberadamente removida dos dados originais.

Fluxo de Trabalho: Pré-treino e Fine-tuning

O processo divide-se em duas etapas fundamentais:

1. **Pré-treino:** O modelo é treinado numa tarefa de Modelação de Linguagem (LM) em grandes *corpora*. Aprende estruturas gramaticais, factos e contextos gerais.
2. **Fine-tuning (Ajuste Fino):** Os parâmetros resultantes servem como inicialização para uma tarefa específica com poucos dados rotulados. O SGD mantém o modelo relativamente perto do mínimo encontrado no pré-treino, o que melhora a generalização.

14.5. Arquiteturas para Pré-treino

Existem três abordagens principais baseadas na estrutura do Transformer:

- **Decoders:** Modelos de linguagem tradicionais. Excelentes para geração, mas não conseguem condicionar palavras no "futuro" devido à máscara causal.
- **Encoders:** Permitem contexto bidirecional (olham para o passado e futuro), sendo ideais para classificação e extração.
- **Encoder-Decoders:** Combinam as vantagens de ambos para tarefas complexas.

14.6. Descodificadores Pré-treinados (Família GPT)

Os modelos GPT (*Generative Pretrained Transformer*) utilizam a arquitetura de descodificador do Transformer para prever a próxima palavra dada a história anterior.

Adaptação para Tarefas a Jusante (*Downstream Tasks*)

Para tarefas de classificação, o modelo utiliza a representação do último estado oculto (h_L):

$$y = \text{softmax}(Ah_L + b)$$

Os parâmetros A e b são inicializados aleatoriamente, enquanto o resto da rede pode ser "congelado" (*frozen*) ou ajustado inteiramente através de retropropogação.

O Modelo GPT Original (2018)

- **Configuração:** 12 camadas de descodificador, estados ocultos de 768 dimensões.
- **Tokenização:** BPE com 40.000 fusões.
- **Dados:** Treinado no *BooksCorpus* (mais de 7000 livros), permitindo a aprendizagem de dependências de longa distância em textos contíguos.

Formatação de Input para Fine-tuning

Para tarefas como Inferência de Linguagem Natural (NLI), os inputs são concatenados utilizando tokens especiais:

[START] Premissa [DELIM] Hipótese [EXTRACT]

O classificador linear é então aplicado à representação final do token [EXTRACT].

14.7. Codificadores Pré-treinados (Encoders)

Ao contrário dos descodificadores, os codificadores beneficiam de um contexto bidirecional total. No entanto, esta bidirecionalidade impede a utilização da modelação de linguagem tradicional (prever a próxima palavra), uma vez que a palavra a prever estaria visível no contexto. A solução é o **Masked Language Modeling (MLM)**.

Masked Language Modeling (MLM)

Esta técnica funciona de forma análoga a um *denoising auto-encoder*:

- **Processo:** Substitui-se uma fração das palavras do input por um token especial `[MASK]`.
- **Objetivo:** O modelo deve prever as palavras mascaradas com base no contexto à esquerda e à direita simultaneamente.
- **Perda:** A função de perda é calculada apenas sobre os tokens mascarados: $p_\theta(x \mid \tilde{x})$, onde $\tilde{x}$ é a versão corrompida do texto.

and output predictions]

14.8. BERT: Bidirectional Encoder Representations from Transformers

Lançado em 2018, o BERT revolucionou o processamento de linguagem natural (NLP) ao mascarar aleatoriamente 15% das palavras de entrada.

Configurações e Treino

Foram lançadas duas versões principais, treinadas no *BooksCorpus* e na Wikipédia Inglesa:

- **BERT Base:** 12 camadas, 768 dimensões, 110 milhões de parâmetros.
- **BERT Large:** 24 camadas, 1024 dimensões, 340 milhões de parâmetros.

Embora o pré-treino seja computacionalmente dispendioso (exigindo TPUs), o *fine-tuning* é prático numa única GPU, seguindo a filosofia: "Pré-treinar uma vez, ajustar muitas vezes".

Limitações dos Encoders

- **Geração:** Codificadores como o BERT não são adequados para tarefas de geração auto-regressiva.
- **Casos de Uso:** São ideais para classificação, etiquetagem de sequências (*tagging*) e extração de informação, onde a bidirecionalidade é uma vantagem crítica.

14.9. Codificador-Descodificador Pré-treinado (T5)

Modelos como o **T5 (Text-to-Text Transfer Transformer)** unificam todas as tarefas de NLP num formato de texto-para-texto, utilizando uma arquitetura completa de codificador-descodificador.

Span Corruption (Denoising)

Em vez de mascarar tokens individuais, o T5 utiliza a corrupção de intervalos (*spans*):

1. Substituem-se sequências de palavras por sentinelas únicas (ex: `<X>`, `<Y>`).
2. O descodificador é treinado para reconstruir apenas os intervalos removidos.
3. **Exemplo:**
   - *Input:* "Thank you `<X>` me to your party `<Y>` week."
   - *Target:* "`<X>` for inviting `<Y>` last `<Z>`"

Capacidades do T5

O T5 demonstrou que a corrupção de *spans* é superior à modelação de linguagem pura em diversas tarefas. O modelo pode ser ajustado para responder a perguntas utilizando o conhecimento factual armazenado nos seus próprios parâmetros (procura de conhecimento "em memória").

14.10. Limitações do Fine-tuning e Eficiência de Parâmetros

Embora o paradigma de pré-treino seguido de *fine-tuning* seja eficaz, ele torna-se impraticável quando lidamos com uma escala massiva de tarefas, domínios (médico, legal, notícias) ou centenas de línguas. O custo de manter uma cópia completa do modelo para cada tarefa é proibitivo.

14.11. Adaptadores (Adapters)

Os adaptadores surgem como uma alternativa eficiente ao *fine-tuning* integral. Em vez de atualizar todos os parâmetros do modelo, inserem-se pequenos módulos específicos para a tarefa entre as camadas do Transformer.

Funcionamento e Vantagens

- **Parâmetros Fixos:** Durante o treino na tarefa a jusante, os parâmetros do modelo pré-treinado original permanecem congelados (*frozen*).
- **Treino Localizado:** Apenas os parâmetros dos adaptadores são atualizados.
- **Modularidade:** É possível partilhar o mesmo modelo base para múltiplas tarefas, carregando apenas os pequenos ficheiros de adaptadores correspondentes.
- **Multilinguismo:** Podem ser utilizados adaptadores de língua e adaptadores de tarefa de forma combinada para transferir conhecimento entre idiomas.

14.12. Aprendizagem com Poucos Exemplos (Few-Shot Learning)

Para cenários onde não existem dados suficientes sequer para um *fine-tuning* leve, modelos de grande escala (como o GPT-3) demonstraram a capacidade de realizar tarefas "na hora" através de **prompting**.

Prompting e In-Context Learning

Esta técnica aproveita a versatilidade dos modelos de linguagem para resolver problemas sem alterar os pesos do modelo. O modelo utiliza o contexto fornecido no *prompt* para inferir a tarefa.

O que os Modelos de Linguagem Aprendem?

Durante o pré-treino, os modelos adquirem diversos tipos de conhecimento que podem ser extraídos via *prompting*:

- **Trivia e Factos:** "O Instituto Superior Técnico localiza-se em [Lisboa], Portugal."
- **Sintaxe e Coreferência:** Identificar a quem se refere um pronome numa frase complexa.
- **Semântica e Sentimento:** Inferir se uma crítica de cinema é positiva ou negativa com base no contexto.
- **Raciocínio e Aritmética:** Completar sequências lógicas ou numéricas (ex: sequência de Fibonacci).

14.13. Conclusão: A Versatilidade dos LMs

O sucesso do *few-shot learning* sugere que os modelos de linguagem pré-treinados funcionam como bases de conhecimento massivas e motores de raciocínio estatístico que podem ser moldados para novas tarefas apenas através da manipulação cuidadosa do texto de entrada.

14.14. Aprendizagem e Otimização de Prompts

Embora o *prompting* seja uma ferramenta poderosa, o design manual de um bom *prompt* é um processo tedioso e os sistemas demonstram ser bastante frágeis e sensíveis a pequenas alterações na escolha das palavras.

Estratégias de Robustez e Automatização

Para mitigar esta sensibilidade, a investigação recente foca-se em aprender os *prompts* de forma automática através de dados de *fine-tuning*:

- **Prompts Discretos:** Envolve a procura automática de sequências de palavras ou tokens que maximizem o desempenho numa tarefa específica. Trata-se de um problema combinatório complexo.
- **Prompts Contínuos (Prompt Tuning):** Em vez de palavras legíveis por humanos, aprendem-se diretamente os vetores de *embedding* que servem de prefixo para a tarefa. Esta abordagem é mais fácil de otimizar via gradiente.
- **Ensembling:** Combinar múltiplos *prompts* e agregar as respostas para aumentar a robustez do sistema.

14.15. Perigos e Riscos dos Grandes Modelos Pré-treinados

Apesar dos sucessos, a utilização de modelos de linguagem de larga escala levanta preocupações éticas e sociais críticas, conforme documentado por Bender et al. (2021).

Preocupações Éticas e Sociais

- **Dados Não Curados:** Muitas vezes, os dados de treino não são representativos da população mundial, apresentando uma visão enviesada da realidade.
- **Eurocentrismo:** Existe um forte domínio da língua inglesa; línguas de baixos recursos estão sub-representadas, o que aprofunda a exclusão digital.
- **Propagação de Enviesamentos:** Os modelos podem reproduzir preconceitos históricos e discriminar minorias presentes nos dados de treino.
- **Privacidade:** Informação privada contida nos dados de treino pode ser memorizada e exposta pelo modelo durante a geração.
- **Conteúdo Tóxico e Alucinações:** A saída do modelo é frequentemente descontrolada, podendo gerar texto ofensivo, tóxico ou informações factualmente incorretas (alucinações) com consequências imprevisíveis.

15. Lecture 12: Deep Generative Models

15.1. Modelação Generativa

Modelar dados de alta dimensão (como imagens) é um desafio central em inteligência artificial. Os modelos generativos profundos procuram aproximar a distribuição de probabilidade dos dados, podendo ser:

- **Aprendizagem Não Supervisionada:** Modelação de $P(x)$.
- **Aprendizagem Supervisionada:** Modelação de $P(x, y)$.

Frequentemente, estes modelos utilizam **variáveis latentes** h para simplificar a distribuição, de tal forma que $P(x) = \sum_h P(x, h)$, onde a distribuição conjunta é mais simples de definir.

15.2. Modelos Baseados em Energia (EBMs)

Estes modelos definem uma distribuição de probabilidade através de uma função de energia $E(x, h; \theta)$:

$$P_\theta(x, h) = \frac{\exp(-E(x, h; \theta))}{Z(\theta)}$$

Onde $Z(\theta) = \sum_{x,h} \exp(-E(x, h; \theta))$ é a função de partição. Maximizar a probabilidade dos dados equivale a minimizar a sua energia. Os principais desafios destes modelos incluem o cálculo de $Z(\theta)$, a inferência do posterior $P_\theta(h \mid x)$ e a amostragem eficiente.

15.3. Máquinas de Boltzmann (BM)

A Máquina de Boltzmann é um modelo baseado em energia sobre vetores binários, contendo variáveis observadas (v) e latentes (h).

- **Função de Energia:** $E(v, h; \theta) = -v^\top Rv - v^\top Wh - h^\top Sh - b^\top v - c^\top h$.
- **Propriedades:** É um aproximador universal de funções de probabilidade discreta e emula a aprendizagem Hebbiana ("neurónios que disparam juntos, ligam-se juntos").

- **Limitação:** Na sua forma geral, a amostragem, a inferência e o treino são computacionalmente muito difíceis.

15.4. Máquinas de Boltzmann Restritas (RBM)

A RBM (ou *harmonium*) é um caso particular onde o grafo é bipartido: não existem conexões intra-camada ($R = 0$ e $S = 0$).

Vantagens e Independência Condicional

Embora a função de partição $Z(\theta)$ continue a ser intratável, a ausência de conexões internas torna as condicionais $P_\theta(h \mid v)$ e $P_\theta(v \mid h)$ tratáveis e fáceis de amostrar:

- As variáveis latentes são condicionalmente independentes dado o vetor visível:

$$P_\theta(h \mid v) = \prod_{j=1}^{M} P(h_j \mid v)$$

- A probabilidade de ativação de uma unidade latente é: $P_\theta(h_j = 1 \mid v) = \sigma(c_j + (Wv)_j)$.

15.5. Variantes Profundas

As RBMs podem ser empilhadas para formar modelos mais complexos e profundos:

- **Deep Belief Networks (DBN):** Modelos híbridos (direcionados e não direcionados) com múltiplas camadas latentes.
- **Deep Boltzmann Machines (DBM):** Modelos totalmente não direcionados com várias camadas de variáveis latentes.

15.6. Redes Geradoras Diferenciáveis

Vários modelos generativos modernos baseiam-se em redes geradoras diferenciáveis. Estas redes utilizam uma função $G(h; \theta)$ que mapeia variáveis latentes h em reconstruções de amostras x ou em distribuições $P_\theta(x \mid h)$. Esta arquitetura é a base para:

- **Variational Auto-encoders (VAE)**
- **Generative Adversarial Networks (GAN)**

15.7. Variational Auto-encoders (VAE)

Os VAEs surgem para resolver problemas onde a evidência $P(x)$ e o posterior $P(h \mid x)$ são intratáveis. A estratégia consiste em combinar a arquitetura de um *auto-encoder* ($x \to h \to \hat{x}$) com os princípios da inferência variacional.

Pressupostos do Modelo

Assume-se que o *prior* $P_\theta(h)$ (ex: Gaussiana padrão) e a condicional $P_\theta(x \mid h)$ são fáceis de computar. No entanto, a marginalização de h para obter $P_\theta(x)$ permanece intratável, exigindo uma aproximação para o posterior.

15.8. Entropia e Divergência de Kullback-Leibler (KL)

Para formular a inferência variacional, utilizamos conceitos da Teoria da Informação:

- **Entropia de Shannon $H(P)$:** Mede a incerteza ou quão próxima da uniforme uma distribuição está. No caso contínuo, é chamada de entropia diferencial.
- **Divergência KL ($KL(P \parallel Q)$):** Mede a diferença entre duas distribuições P e Q. É sempre não-negativa e vale 0 se, e apenas se, $P = Q$. Não é simétrica.

15.9. Evidence Lower Bound (ELBO)

O ELBO é o conceito central da inferência variacional. Como maximizar a evidência $\log P_\theta(x)$ diretamente é impossível, maximizamos o seu limite inferior.

$$\log P_\theta(x) = \text{ELBO}(Q) + KL(Q(h) \parallel P_\theta(h \mid x)) \geq \text{ELBO}(Q)$$

Maximizar o ELBO é equivalente a minimizar a divergência KL entre a distribuição aproximada $Q(h)$ e o posterior verdadeiro $P_\theta(h \mid x)$.

Decomposição do ELBO

O ELBO pode ser escrito como:

$$\text{ELBO}(Q) = \mathbb{E}_{Q(h)}[\log P_\theta(x \mid h)] - KL(Q(h) \parallel P_\theta(h))$$

- **Primeiro termo (Verosimilhança esperada):** Encoraja $Q(h)$ a colocar massa em valores de h que expliquem bem os dados observados x.
- **Segundo termo (Regularização):** Encoraja $Q(h)$ a manter-se próximo do *prior* $P_\theta(h)$.

15.10. Aproximação Mean Field

Para tornar $Q(h)$ tratável, a família de distribuições mais comum é a **Mean Field Approximation (MFA)**, que assume que todas as variáveis latentes são independentes entre si:

$$Q(h) = \prod_i Q(h_i)$$

15.11. Inferência Variacional Amortizada

Otimizar $Q(h)$ individualmente para cada exemplo x é computacionalmente caro. A **Inferência Variacional Amortizada** resolve este problema:

- **Codificador (Encoder):** Em vez de otimização direta, utiliza-se uma rede neuronal com parâmetros partilhados ϕ para definir $Q_\phi(h \mid x)$.
- **Eficiência:** Para cada exemplo, basta uma passagem em frente (*forward pass*) no codificador para obter os parâmetros da distribuição variacional.

15.12. Exemplo: Bernoulli Multivariada com Variáveis Latentes Contínuas

O modelo proposto por Kingma e Welling (2013) é uma das implementações mais influentes de VAEs, combinando variáveis latentes contínuas com observações discretas.

- **Prior:** Assume-se uma distribuição Gaussiana padrão: $P_\theta(h) = \mathcal{N}(h; 0, I)$.
- **Condicional (Descodificador):** Uma Bernoulli multivariada onde as probabilidades são geradas por um MLP (Perceptron Multicamada):

$$P_\theta(x \mid h) = \prod_{i=1}^{D} \sigma(f_i(h;\theta))^{x_i}(1 - \sigma(f_i(h;\theta)))^{1-x_i}$$

- **Posterior Aproximado (Codificador):** Como o posterior verdadeiro é intratável, utiliza-se uma rede de inferência que produz a média μ e a variância σ^2 de uma Gaussiana:

$$Q_\phi(h \mid x) = \mathcal{N}(h; \mu(x;\phi), \sigma^2(x;\phi))$$

15.13. Gradientes dos Parâmetros

Para treinar o modelo, precisamos de calcular os gradientes do ELBO em relação aos parâmetros do descodificador (θ) e do codificador (ϕ).

Gradiente em relação a θ (Rede de Geração)

O cálculo é direto devido à linearidade do valor esperado:

$$\nabla_\theta \text{ELBO}(\phi;\theta) = \mathbb{E}_{Q_\phi(h|x)}[\nabla_\theta \log P_\theta(x,h)]$$

Este termo pode ser facilmente aproximado através de amostragem de Monte Carlo simples, utilizando amostras h_i geradas pelo codificador.

Gradiente em relação a ϕ (Rede de Inferência)

O gradiente em relação aos parâmetros de inferência é mais problemático:

$$\nabla_\phi \text{ELBO}(\phi;\theta) = \mathbb{E}_{Q_\phi(h|x)}[(\log P_\theta(x,h) - \log Q_\phi(h|x))\nabla_\phi \log Q_\phi(h|x)]$$

- **Problema:** Esta formulação (semelhante ao algoritmo REINFORCE em RL) apresenta uma variância extremamente elevada nas estimativas de Monte Carlo.
- **Consequência:** O treino torna-se instável e lento, exigindo técnicas de redução de variância.

15.14. Derivação do Gradiente da Rede de Inferência

A derivação utiliza a regra do log-derivado ($\nabla_\phi Q_\phi = Q_\phi \nabla_\phi \log Q_\phi$) e o facto de que a soma das probabilidades é 1 (logo, o gradiente da soma é 0).

A expressão final mostra que o gradiente depende do produto entre uma "recompensa" (a diferença entre a log-probabilidade conjunta e a log-probabilidade variacional) e o gradiente da log-densidade da distribuição variacional.

$$\nabla_\phi \text{ELBO} = \sum_h [\log P_\theta(x,h) - \log Q_\phi(h|x)]\, \nabla_\phi Q_\phi(h|x)$$

15.15. O Truque de Reparametrização (Reparameterization Trick)

Como vimos, a dificuldade reside no gradiente em relação a ϕ, cujas estimativas de Monte Carlo apresentam uma variância elevada. O truque de reparametrização, introduzido por Kingma e Welling (2013), permite reescrever o processo de amostragem para que o gradiente possa fluir através da rede de inferência via retropropagação padrão.

Mecânica do Truque

Em vez de amostrar h diretamente de $Q_\phi(h \mid x)$, o que quebraria a diferenciabilidade, utilizamos uma variável aleatória auxiliar ϵ com uma distribuição fixa:

1. **Amostragem Externa:** Amostra-se ϵ de uma distribuição simples, tipicamente $\epsilon \sim \mathcal{N}(0, I)$.
2. **Função Determinística:** Obtém-se h como uma transformação de ϵ, x e dos parâmetros ϕ:
$$h = g_\phi(\epsilon, x) = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon$$

Desta forma, para qualquer função f, o valor esperado pode ser aproximado por:

$$\mathbb{E}_{Q_\phi(h|x)}[f(h)] \approx \frac{1}{N} \sum_{i=1}^{N} f(g_\phi(x, \epsilon^{(i)}))$$

Aplicabilidade

Este truque é aplicável a diversas famílias de distribuições contínuas:

- Exponencial e Gaussiana.
- Famílias de localização-escala (*location-scale*).
- Log-normal.
- Para variáveis discretas, embora mais complexo, utiliza-se o truque de **Gumbel-softmax**.

15.16. A Estrutura Final do VAE

Combinando estes elementos, o Variational Auto-Encoder (VAE) funciona da seguinte forma:

- **Codificador (Encoder):** Computa os parâmetros da distribuição variacional, $\mu_\phi(x)$ e $\sigma^2_\phi(x)$.
- **Espaço Latente:** Realiza a amostragem de h usando o truque de reparametrização.
- **Descodificador (Decoder):** Recebe h e reconstrói o input, maximizando a verosimilhança $P_\theta(x \mid h)$.
- **Função de Perda:** Minimiza-se o negativo do ELBO, que equilibra o erro de reconstrução e a divergência KL em relação ao *prior*.

15.17. O que Representam as Variáveis Latentes?

Uma propriedade notável dos VAEs é a sua capacidade de realizar **manifold learning** (aprendizagem de variedades). Ao treinar simultaneamente o codificador e o gerador, o modelo é forçado a organizar o espaço latente de forma estruturada e contínua.

Descoberta de Fatores de Variação

O modelo aprende frequentemente eixos independentes que correspondem a características semânticas dos dados sem supervisão explícita. Por exemplo, em imagens de rostos, o VAE pode isolar fatores como:

- O ângulo de rotação da cabeça.
- A expressão emocional (ex: de neutro para sorriso).

15.18. Problemas com VAEs

Apesar da sua robustez, os VAEs sofrem frequentemente de um fenómeno conhecido como **colapso do posterior** (*posterior collapse*).

- **O Fenómeno:** Se o descodificador for demasiado potente (por exemplo, uma RNN forte), o modelo pode aprender a ignorar as variáveis latentes. Nestes casos, a distribuição variacional torna-se idêntica ao *prior* e o termo de reconstrução ignora h.
- **Mitigação:**
  - Aplicar *annealing* (aumento gradual) ao peso da divergência KL no ELBO.
  - Utilizar perdas auxiliares para forçar a codificação de informação útil.
  - Combinar inferência estocástica com inferência amortizada.

15.19. Porquê ir além da Máxima Verosimilhança?

Até agora, todos os modelos (RBMs, VAEs) focaram-se em maximizar a verosimilhança $P(x)$. No entanto, este critério tem limitações práticas:

- Tende a produzir imagens "baças" ou desfocadas (*blurry*).
- O modelo tenta cobrir todos os modos dos dados, o que pode resultar numa média visual pouco nítida.

15.20. Generative Adversarial Networks (GANs)

Introduzidas por Goodfellow et al. (2014), as GANs abandonam a rede de inferência e propõem um jogo entre duas redes:

1. **Gerador (G):** Tenta gerar dados que pareçam reais a partir de ruído h.
2. **Discriminador (D):** Atua como um crítico que tenta distinguir entre dados reais e dados "falsos" produzidos pelo gerador.

É análogo a um Teste de Turing, onde o objetivo do gerador é tornar-se indistinguível da realidade.

O Jogo Minimax

O treino é formulado como um problema de ponto de sela (*saddle point*):

$$\min_G \max_D V(D, G) = \mathbb{E}_{P_{data}(x)}[\log D(x)] + \mathbb{E}_{P_\theta(h)}[\log(1 - D(G(h)))]$$

O gerador ideal minimiza a **Divergência de Jensen-Shannon** entre a distribuição dos dados reais e a distribuição gerada.

15.21. Wasserstein GANs (WGANs)

As GANs originais são notoriamente instáveis. A WGAN propõe uma função de custo baseada na **Distância de Wasserstein** (ou *Earth Mover's Distance*):

- **Estabilidade:** Melhora a convergência e mitiga o problema de **mode collapse** (onde o gerador produz sempre o mesmo tipo de amostra).
- **Restrição:** Exige que o gradiente do discriminador seja limitado, o que na prática é feito através de *gradient clipping*.

15.22. Treino de GANs

O treino utiliza SGD, alternando entre atualizações dos parâmetros do gerador (θ) e do discriminador (D).

Ressalvas Importantes

- **Convergência:** Não existem garantias de convergência para um equilíbrio global.
- **Dificuldade:** A otimização é instável; o discriminador pode tornar-se demasiado forte rapidamente, impedindo o gerador de aprender (problema de gradientes nulos).

15.23. Denoising Diffusion Models: Ideia Fundamental

Os modelos de difusão baseiam-se na premissa de que aprender a remover ruído (*denoise*) é equivalente a aprender um modelo de probabilidade dos dados $P_\theta(x)$. O processo divide-se em duas fases: a difusão direta (adição de ruído) e a difusão inversa (geração).

Difusão Direta (Forward Diffusion)

Nesta fase, adicionamos ruído gaussiano aos dados de forma incremental ao longo de T passos:

- **Transição:** $x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t}\epsilon$, onde $\epsilon \sim \mathcal{N}(0, I)$.
- **Preservação da Variância:** Se definirmos $\text{var}(x_0) = 1$, então a variância mantém-se unitária em todos os passos ($\text{var}(x_t) = 1$).
- **Kernel de Difusão:** É possível saltar diretamente de x_0 para qualquer passo t sem iterar:

  $$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$$

  Onde $\alpha_t = 1 - \beta_t$ e $\bar{\alpha}_t$ é o produto acumulado de $\alpha_1 \dots \alpha_t$.

15.24. Aprendizagem Generativa via Remoção de Ruído

O objetivo central é treinar uma rede neuronal ϵ_θ para estimar o ruído que foi adicionado a uma imagem.

- **Input da Rede:** O passo de tempo t e a imagem ruidosa x_t.
- **Output da Rede:** Uma estimativa do ruído original $\hat{\epsilon}$.
- **Convoluções:** Na prática, a rede ϵ_θ é frequentemente uma U-Net, que processa a imagem através de camadas convolucionais para capturar estruturas em múltiplas escalas.

15.25. Algoritmos Forward e Backward

O processo de geração (amostragem) inverte a lógica da difusão direta.

Amostragem (Processo Inverso)

Para gerar uma nova imagem, o modelo segue estes passos:

1. Começar com ruído puro $x_T \sim \mathcal{N}(0, I)$.
2. **Denoising Sequencial:** Para cada passo de T até 1:
   - Estimar o ruído presente em x_t usando a rede: $\hat{\epsilon} = \epsilon_\theta(x_t, t)$.
   - Subtrair a estimativa de ruído $\hat{\epsilon}$ de x_t com o escalonamento apropriado para obter x_{t-1}.
3. O resultado final x_0 será uma amostra da distribuição de dados aprendida.

Noise Schedule

A escolha de β_t (o *noise schedule*) define a rapidez com que a imagem se transforma em ruído. Um escalonamento linear é comum, mas existem variantes (como o cosseno) que podem melhorar a qualidade da geração.

16. Lecture 13: Uncertainty, Explainability, and Fairness

16.1. Incerteza em Machine Learning

A incerteza é crucial para que os modelos "saibam o que não sabem", permitindo decidir se uma previsão deve ser inspecionada ou se é seguro utilizá-la em aplicações de alto risco.

Tipos de Incerteza

Existem dois tipos principais de incerteza que compõem a incerteza preditiva total ($U_{total} = U_{al} + U_{ep}$):

- **Incerteza Aleatória (Dados):** Refere-se ao ruído intrínseco nos dados, como medições imprecisas, anotações ruidosas ou dados parcialmente observados. É considerada um erro irredutível.

- **Incerteza Epistémica (Modelo):** Refere-se à incerteza no próprio modelo devido à falta de conhecimento ou dados de treino insuficientes. Inclui:
  - **Covariate Shift:** Mudança na distribuição das *features* $p(x)$, mantendo $p(y|x)$.
  - **Concept Shift:** Mudança na condicional $p(y|x)$, mantendo $p(x)$.
  - **Label Shift:** Mudança na distribuição das etiquetas $p(y)$, mantendo $p(x|y)$.
  - **Open-set recognition:** Surgimento de novas classes no momento de teste.

Quantificação e Avaliação

A incerteza pode ser medida através da entropia:

- **Classificação:** $\mathcal{H}(p(\hat{y}|x)) = -\sum_{\hat{y}\in\mathcal{Y}} p(\hat{y}|x) \log p(\hat{y}|x)$
- **Regressão:** $\mathcal{H}(p(\hat{y}|x)) = -\int_{\hat{y}\in\mathcal{Y}} p(\hat{y}|x) \log p(\hat{y}|x) d\hat{y}$

Para obter a distribuição $p(\hat{y}|x)$, utilizam-se métodos como:

- **Bayesian Neural Networks (BNN):** Assumem um *prior* sobre os pesos e tentam computar o posterior $p(W|X, Y)$, frequentemente via inferência variacional.
- **Deep Ensembles (DE):** Treinam-se múltiplos modelos com diferentes sementes ou hiperparâmetros para aproximar a distribuição.
- **Monte Carlo Dropout (MCD):** Aplica-se *dropout* durante o teste, realizando múltiplas passagens para estimar a incerteza.

Calibração

A incerteza deve ser representativa da frequência com que o modelo erra. O **Expected Calibration Error (ECE)** mede esta correspondência dividindo as previsões em *bins* de confiança e calculando:

$$ECE = \frac{1}{M} \sum_{m=1}^{M} |acc(B_m) - conf(B_m)|$$

16.2. Explicabilidade (Explainability)

A explicabilidade procura fornecer justificações compreensíveis para as decisões dos modelos, sendo vital para aplicações médicas, financeiras ou legais.

Explicabilidade Intrínseca vs. Post-hoc

- **Intrínseca:** Modelos transparentes por natureza, como árvores de decisão, modelos lineares (GAMs, GLMs) e redes Bayesianas.
- **Post-hoc:** Extração de explicações de modelos "caixa-preta":
  - **Global:** Explica o comportamento do modelo como um todo.
  - **Local:** Explica decisões em inputs individuais.

Métodos de Atribuição Local

- **Baseados em Gradiente:** Revelam quais regiões do input afetam mais o output (ex: Integrated Gradients, SmoothGrad, GradCAM).
- **Baseados em Atenção:** Interpretam os pesos de atenção como importância das *features*. Nota: Existe debate se "atenção é explicação" devido à baixa correlação com gradientes nalguns casos.
- **Agnósticos ao Modelo:**
  - **LIME:** Ajusta um modelo linear local numa vizinhança perturbada do input.
  - **MICE (Counterfactuals):** Explica a decisão através de edições mínimas que alterariam a previsão (ex: "Se o filme fosse mais curto, a nota seria positiva").

16.3. Equidade (Fairness) e Ética

A IA ética deve considerar discriminação, acessibilidade, privacidade e sustentabilidade.

Tipologia de Enviesamento (Bias)

- **Sample Bias:** Dados de treino não representativos.
- **Association Bias:** Correlações espúrias nos dados de treino (ex: associar "médico" a homens).
- **Discrimination Bias:** Dados enviesados contra demografias específicas (raça, género).

Exemplos clássicos incluem algoritmos de recrutamento que discriminam mulheres ou sistemas de justiça criminal (COMPAS) com taxas de erro superiores para pessoas negras. Em LLMs, observam-se analogias enviesadas nos *embeddings* (macho-médico vs. fêmea-enfermeira) e propensão para completar frases sobre muçulmanos com temas violentos.

Correção de Bias

- **Pré-treino:** Uso de *datasets* diversos e *Counterfactual Data Augmentation* (CDA).
- **Treino:** Termos de perda que restringem informação demográfica nos vetores.
- **Pós-treino:** Projeção de representações para fora de dimensões de género.

16.4. Sustentabilidade e Privacidade

- **Sustentabilidade:** O crescimento exponencial de modelos (de ELMo com 94M a Megatron-Turing com 530B parâmetros) levanta sérias questões sobre o consumo energético.
- **Privacidade:** A anonimização pode ser insuficiente se SPI (*Sensitive Personal Information*) puder ser inferida pela combinação de dados. LLMs podem sofrer fugas de dados tanto do treino como do contexto do utilizador.